

MANGAL skillハンドブック

生成: 2026-07-15 / 収録 25 skill (.claude/skills/*/SKILL.md 全文)
再生成: python scripts/_gen-skill-handbook-pdf.py

skill	概要(SKILL.md冒頭見出し)
backward-distill	後退蒸留して <年>
band-intruder-fix	激マン型(帯混入)是正 (= トリガー「帯混入直して」「激マン型見て」)
cf-analytics	Cloudflareアクセス解析 (= 2026-07-09 トークン・2026-07-10 script/skill化+Web Analytics対応)
daily-distill	日次蒸留 (= 2026-07-06 全面改訂: 楽天予約が支柱に)
diff-deploy	差分反映して (= 差分ビルドエンジン)
display-bug-triage	表示不具合の切り分け (= スクショ/「〜がおかしい」報告を受けたら)
enrich-catch-synopsis	キャッチ/詳細エンリッチ (= トリガー「エンリッチして」「キャッチ詳細作って」)
external-data-access	外部データ照会 (= 楽天・NDL・キャッシュ)
gemini-genre-audit	ジャンル検品 (= トリガー「ジャンル検品して」「Gemini検品して」/ 裁定は「**検品裁定して**」。2026-07-14 ユーザ設計)
idle-run	アイドル運転 (= トリガー「アイドル運転して」/ 停止「やめて」。2026-07-14 ユーザ設計)
isbn-dup-cleanup	ISBNダブリ潰し (= 同ISBNが複数ページに載る事故の解体)
kobo-covers	Kobo書影補完 (= トリガー「Koboして」「書影補完して」「Kobo続けて」)
long-job-ops	長時間ジョブの運転法
monthly-distill	月次蒸留して
new-manga-register	新規追加 / 新刊入れて
percase-fix	per-case是正 (= イアラ式。ユーザが作品名+Wiki/NDLリンクを出したら)
productionize-drafts	本番化して (= 確認済み予約ドラフトの本番化。2026-07-09 ユーザ設計)
prodwait-preview	本番待ちをテスト環境へ (= 2026-07-10 確立・script化)
reflect-targeted	反映して (= targeted反映)
tameshiyomi-harvest	試し読み拾って (= BookLive title_id 収集。2026-07-12 設計、同日★全巻展開を追加)
test-deploy	テスト環境に出して (= preview投入)
tinyfish	TinyFish (= WebFetch補完。2026-07-08 導入・2026-07-10 skill化)
volgap-audit	巻抜け仮想 / 巻系監査
weekly-distill	週次蒸留 (= フルビルド・フルアップ)
wiki-distill	Wiki蒸留して

backward-distill

後退蒸留して <年>

```
---
name: backward-distill
description: 後退蒸留して <年>=過去年をNDLで発見→仕分け→AI
worksheet→preview新規頁生成。掲載ゲート=必須メタ完備+楽天書影
---
```

トリガー語: **「後退蒸留して 2024」** のように年付き。

手順 (= scripts/_distill_backward.py の3段)

1. `python scripts/\_distill\_backward.py <年> --discover` — NDL
live(NDC726.1・月分割・**1.2秒/req・429=即中断**)。逐次保存=中断再開可
2. `--plan` — 仕分け: 漫画性フィルタ(_promote_drop_patterns.py) → A route(既存作の巻=種4化) / B route(新規作) →
楽天cache enrich → **掲載ゲート** (必須メタ: title/kana/romaji/authors/year/status/demographic/genre≥1 +
楽天書影v1) → AI worksheet + **欠落表**
3. worksheet 記入(AI): 新規登録protocol厳守(下記) → `--emit` — preview新規頁生成(検証: closed
vocab/slug衝突/demographic enum/Zodミラー/日付pad/publisher master正規化/L1 booksGenreId/L2 tags)

新規登録protocol (= 順番固定・CLAUDE.md「新規登録 protocol」)

- ①全巻回収が先(単巻先行禁止) ②題確定=NDL×楽天突合(勝手命名禁止) ③ヨミ/著者=NDL典拠(不明=報告) ④一括登録
⑤enrich=1巻基点・ネタバレ禁止 ⑥作れない物=欠落表(捏造して載せない)

NEVER

- ・429連打・捏造・単巻先行・closed vocabulary外のgenre・最終巻あらずじ丸写し
- ・被覆台帳=distill-coverage.json を更新し忘れない
- ・本番化はユーザ確認後(preview で見せて GO を待つ)

済み範囲

2022-2024は取得済(台帳確認)。重複は series_key/ISBN dedup が守るが、走る前に `--plan` の差分レポートで確認。

band-intruder-fix

激マン型(帯混入)是正 (= トリガー「帯混入直して」「激マン型見て」)

name: band-intruder-fix

description:

激マン型/帯混入是正=少数ISBN帯×日付逆行の混入巻を検出→NDL正史×楽天照合で真巻にスワップ。月次サニティ級

長期連載の中に**少数ISBN帯で日付が後続majority巻より後**の巻が紛れる型(=別版/コンビニ本/別作の混入)を検出し、NDL正史×楽天照合の二段ゲートで真巻にスワップする。96頁/170巻で実証(2026-07-05・激マン!で発覚)。

検出

```
python scripts/_audit-band-intruders.py
```

- ・各頁の各editionで majority帯(≥50%) を基準に、**少数帯の巻で「その巻の発売日 > 後続majority巻の発売日」**を混入と判定。
- ・出力 `docs/production-diagnostics/band-intruders.tsv` (slug/edition/vol/intruder_isbn)。

是正 (= NDL→楽天の二段ゲート。ユーザ裁定=「ISBN有は潰せる。NDL回してから楽天照合が確度・効率」)

1. **NDL harvest** (該当作をtitle+creatorで全件・★ページング必須):

```
python scripts/_band-intruder-ndl-fetch.py # 初回(200件/req)
```

★NDL SRUIは1リクエスト最大200件。大物(009/おそ松=数百版)は`startRecord`で全ページ取得しないと原版を取りこぼす([external_data_access])のNDLページング)。取りこぼし時はページング版で再harvest。

★BibResourceが2断片に割れ、原版のISBNとvol/dateが泣き別れる型あり→結合処理(merge_fragments)必須。

2. **スワップ生成** (NDL全帯U楽天直接の候補から、楽天題完全一致+datefitで確定):

```
python scripts/_band-intruder-swap.py # dry-run
```

```
python scripts/_band-intruder-swap.py --apply # volume-exclude(混入除去)+種4(真巻追加)
```

- ・★帯は**必須にしない** (出版社移籍作=EAT-MAN[メディアワークス→MF]があるため)。ゲート=楽天題base完全一致+年代整合。
- ・混入ISBN→`volume-exclude.yml` (slug単位)、真巻→`volumes-supplement.yml` (種4・db-v2逆引きでseries_key確定)。

3. **反映**: touchedを `\_reflect-targeted.py --only ... --push`。

4. **検証**: 反映後に対象頁の帯統一/日付逆行0を数字で確認。通らない残り(候補なし/複数候補/裏取り不可)は `band-intruders-manual.tsv` へ。

ゲート(fail-closed)

- ・楽天題baseが頁題と完全一致するISBNのみ採用(別作混入を防ぐ)。
- ・既にどこかで表示中(isbn-page-index)のISBNは候補外=merge案件として別扱い。
- ・複数候補/裏取り不可はabortしてworklist(推測でスワップしない [[feedback_accuracy_is_the_goal]])。

実測

第1波(帯厳格)9件→ゲート再設計(帯を参考格下げ)第2波67件=計76件スワップ。残103件=ISBN無古典/複数版でworklist。月次サニティで再走可(新たな混入型のsignal)。

cf-analytics

Cloudflareアクセス解析 (= 2026-07-09 トークン・2026-07-10 script/skill化+Web Analytics対応)

name: cf-analytics

description: アクセス解析して=Cloudflare解析2系統(report=Workerリクエスト/エラー・web=訪問者/人気ページ/国)。必ず
_cf-analytics.py から(GraphQL再実装禁止)

トリガー語: **「アクセス解析して」**(「アクセスどう?」「人気ページは?」等も同義)。

やること (2系統・質問に合わせて選ぶ)

```
python scripts/_cf-analytics.py web --days 7 # ★人間の訪問者: 閲覧/訪問/人気ページ/国/流入元
python scripts/_cf-analytics.py report --days 7 # Workerインフラ: 総req/エラー率(クローラ込み=配信健康)
python scripts/_cf-analytics.py verify # トークン生存確認(失敗時の切り分け)
```

- ・「アクセスどう?」= **web が主役** (訪問者視点)、report は配信健康(エラー率)の補助。
- ・endpoint/認証/GraphQL(workersInvocationsAdaptive+rumpageloadEventsAdaptiveGroups)の正は**scriptに封じ込め済み=再実装しない**。siteTag等の定数もscript内。
- ・キー: `env` の `CLOUDFLARE\_API\_TOKEN` (Analytics Read。★絶対commitしない)。RUM RESTは403=scope外だがGraphQL rumは通る(実証済)。

読み方の規律 (= 数字を誤読して報告しない)

- ・**web(ビーコン計測)=人間**:
JS実行ブラウザのみ、bot/クローラは原則含まれない。設置=2026-07-05(自動セットアップ)以降のみ。
- ・**report(Worker)=機械込み**: requests≠訪問者(1頁=複数ファイル+クローラ支配。波が来ると日10万超/引くと日2千台)。「訪問者N人」はwebの数字でのみ言う。
- ・エラー率(report)平常0.01%未満。急増=配信障害signal。
- ・流入元「(直接)」=referrerなし(ブックマーク/アプリ/一部ブラウザのプライバシー設定)。

使いどころ

- ・週次蒸留後の健康確認(reportのエラー率)・「最近どう?」への即答(web)。
- ・人気ページ(web)は discovery/ランキング戦略の実データ源として蓄積中(2026-07-10時点: 週66訪問・Google流入が出始め)。
- ・定期監視はしない(規模が意味を持つまで=ユーザ判断 2026-07-09)。

関連

- ・事実の記録=memory [[cloudflare_analytics_access]] / 本番構成=memory [[deploy_environments_state]]

daily-distill

日次蒸留 (= 2026-07-06 全面改訂: 楽天予約が主柱に)

name: daily-distill

description: 日次蒸留 — 2本立て: A=楽天予約ハーベスト(未来の新刊・カレンダーの餌)/B=NDL新着回収(過去の納本)。+NDL照合キュー消化+カレンダー更新。トリガー「日次蒸留して」。

****2本立て****: A=楽天予約ハーベスト(未来~当月の新刊。カレンダー/新刊棚の唯一の未来供給源) / B=NDL新着回収(納本済み過去分)。毎日ではなくてよい(間隔が空いても窓が自動で広がり取りこぼさない)。理想運用: 日次蒸留して→(previewで)確認→週次蒸留して=週1本番更新。

NEVER(禁止) ★2026-07-09 ハードニング: 下記4つは今日全部踏んだ実害。飛ばすな

- ・ **★**フルharvest処理禁止=必ず増加分だけ****: `preorders-latest.jsonl` を丸ごとclassifyするな。 ****必ず** `preorders-prev.jsonl` との差分(新ISBN=fresh)に絞ってから ****classify/生成する(下A0)**。フル処理=昨日以前のbacklogを全部「新規」に水増しする(実害2934件)。
- ・ **★**過去ドラフト再カウント禁止****: 増加分(ISBN fresh)でも、 ****前回previewドラフト化したが未promoteの作品****が後続巻の新ISBNで再登場する。`.cache/preorders/drafts` (過去draft)の ****題(base正規化)と突合して除外****してから生成(下A0)。[[daily_distill_classifier_gate]]。
- ・ **★**ヨミ捏造禁止(厳守)****: title_kana=****楽天titleKanaのみ****。楽天に無ければ ****題流用/生成読み/巻番号混入で捏造するな→hold****(NDL照合キュー行き)。 ****title_kanaに漢字が1字でも入る=捏造=即hold****(gen-midfillはゲート済、生成物は必ず漢字混入チェック)。
- ・ ****429/throttle即中断****(NDL・楽天とも1.1~1.3s/req厳守。リトライ連打禁止)。
- ・ ****捏造禁止****: ヨミ/著者/genreを推測で埋めない。ヨミ=楽天仮確定+NDL照合キュー(下記)が正規ルート。
- ・ ****単巻先行登録禁止****: 途中巻でページ無し(④)は全巻回収が成立した作品だけドラフト化。
- ・ ****②③④は必ずpreview先行****(B裁定)。ユーザ確認GOなしに本番化しない。 ****previewは"今回のみ"*****(増加分-過去draft-捏造kana)に絞る。ユーザは「前回見た」に敏感で正確=水増しを即見抜く。
- ・ **★**1回のAPIで全フィールド捕捉****(2026-07-09 ユーザ指摘 [[acquire_all_obtainable_info]]): 楽天API/harvestは書影だけでなく ****itemCaption(あらすじ=genre/catch/synopsis元)/booksGenreId/itemPrice/subTitle/affiliateUrl**** を返す。 ****書影だけ取って捨てるな****。captionを`\_preorder\_draft.rakuten\_caption`に保存→genre(master32・provisional)/catch/synopsissを生成(caption有れば発売前でも付く)。書影のためにAPIを叩くなら同じ応答からcaptionも必ず取る。
 - ・ genre=closed vocabulary(master32)のみ+provisional。catch/synopsis=skill enrich-catch-synopsis の規律。
 - ・ **★**改善は即興でなくskill/scriptに焼く****: 実行中に穴を見つけたら手動で継ぎ足すな。scriptにゲート追加→commitしてから進む(即興は次回消える+報告が水増しでブレる)。

★毎日の実行手順 (= runbook・この順で回す。2026-07-09 確立。ツール把握用)

前提: `git status` clean。楽天/NDL認証env有り。

```
| # | やること | ツール(コマンド) |
|---|---|
| 1 | 予約harvest | `python scripts/_rakuten-preorder-harvest.py` |
| 2 | ★**増加分に絞る** (必須) | `python scripts/_preorder-increment.py` ← prev差分+過去draft除外。飛ばすと水増し |
| 3 | 分類 | `python scripts/_preorder-classify.py` |
| 4 | ①続巻→種4+反映 | `python scripts/_preorder-apply-zokkan.py` → `python scripts/_reflect-targeted.py --only <touched> --push` |
| 5 | ②③新作ドラフト | `python scripts/_preorder-gen-preview.py new1a` ; `new1b` |
| 6 | ④途中巻ドラフト | `python scripts/_preorder-gen-midfill.py` |
| 7 | genre付与(★worksheet化 2026-07-10) | `python scripts/_preorder-genre-worksheet.py --emit` → Alがworksheetのgenres[]にmaster32キー記入(確信なければ空) → `--apply` (master32外=abort・純粹追加・provisional自動)。caption無し頁は先に `python scripts/_preorder-capture-captions.py` |
| 8 | Cヨミ照合 | `python scripts/_verify-kana-pending.py --limit 200` |
| 9 | **検査** (下記チェックリスト) | 欠け>0なら原因調査 |
| 10 | ★**prev確定** (処理完了の宣言) | `python scripts/_preorder-increment.py --commit-prev` ← full→prev昇格。 **これ以外の方法でprevを触るな**。飛ばすと次回差分が壊れる |
| 11 | 索引+暦+push | `python scripts/_build-list-index.py .preview-data/manga .preview-data` ; `python scripts/_build-calendar.py .preview-data/manga public/calendar <当月>` ; commit+push |
| 12 | B NDL新着(任意) | `python scripts/_distill_daily.py --discover` → `--plan` → `--emit` |
```

◆ ★生成後チェックリスト (= 全部2026-07-09に実際踏んだ罫。毎回数える)

- `**kana純カタカナ**`(漢字/ひらがな0)。楽天ヨミ無し/汚染は**捏造せずhold**が効いているか
- `**slug=辞書装置**`(`\_slug\_kana\_lib` 経由)・英語綴り(summer-blend等)。pykakasi再発明でない。英語出ない語は`ka takana-english.yml`に追加
- `**全巻に発売日**`(回収先行巻=種2 db-v2から引く)+**書影** (実URL=harvest/covers seed/楽天API。★構築禁止)
- `**genre**=captionからprovisional・master32のみ`(捏造なら空)
- `**作者kana**=楽天authorKana由来 or 空`(捏造なし。漢字混入0を確認)
- `**preview=今回のみ**`(増加分・前回draft混入0・捏造kana hold済)
- `**索引skip 0**`(Zod検証で落ちる頁=検索に載るが404)
- `**索引衛生**`python scripts/_audit-index-hygiene.py data``(cover slim全行/スキーマ/head/alt。2026-07-14新設ゲート)
- `**`--commit-prev` 実行済み**`(処理完了後のみ。件数確認だけの日は実行しない)
- `★**slug-gate-pending.tsvを確認**`(2026-07-14内蔵のヨミ一致ゲート: 装置が題を誤読した疑い=剣聖ken-hijiri型がここに落ちる。行があれば当該slugを手裁定=ヨミ基準で正す。裸巻数読み尾(アカルイミライニ型)は`clean\_kana(base付き)`が自動トリム済=チェック不要)
 - `★**slug-katakana-pending.tsvを確認**`(2026-07-14: 辞書に掛からずカナ転写したカタカナ語=「自動で決められない箇所」の簿記。lolly/blue-bird-reader型はここから拾ってユーザ報告→裁定。和製語(ママ/クズ/メシ等)はヘボンで正しい=報告不要。外来語らしきもの・公式英字がありそうな物だけWeb裏取り(公式英字>ユーザ裁定>ヘボン維持)。処理済み行は消してよい。一括監査=`\_audit-slug-katakana.py <dir>`)
 - `★**題末尾の巻表示掃引**`(2026-07-14 サムライトルーパー型: 題が「～上」等で終わる新draftは題/kana(ジョウ)/romaji(jou)から剥離し`volumes[0].volume\_label`へ退避。検出=`re.search(r"([ \s ]+|[ \s ]\*[ ( ) (上|下|中|前編|後編) ])?\$", title)`を新規分に掛ける=裸型「～上」と括弧型「～(上)」の両方(ムジナの城型 2026-07-14に8件))
 - `★**「Vol.N」/「仮」を含む新規題は再編集本疑い**`(むこうぶちVol.5仮型 2026-07-14: コンビニ廉価のテーマ別再編集シリーズが仮題のせいで1巻新規としてすり抜けた。`\_lookup.py --title "題 Vol"`で同型Vol.1..N-1の存在と副題(テーマ名)を確認→該当なら drop。本編が本番に既存(_exists.py --title)なのに別レーベルVol.Nが来た場合は特に疑う)

◆ ★今日の学び(runbookに焼いた恒久修正・再発防止)

- 増加分ゲート(過去draft再カウント防止) / kana捏造hold / slug辞書装置 / 発売日=種2引き当て / 書影=実URL(構築禁止) / 1API全フィールド捕捉(caption→genre) / preview今回のみ。

A0. ★増加分ゲート (= 2026-07-09 必須。ここを飛ばすと全部水増しになる)

harvest後・classify前に**必ず**増加分に絞る。`\_preorder-increment.py` (下記機能を1本化):

`python scripts/_preorder-increment.py # ①latest-prev差分(新ISBN) ②過去draft題を除外 → classified.json は増加分のみ機能(script化して飛ばせなくする):`

1. `**fresh = preorders-latest - preorders-prev**`(ISBN差分)。prev無しなら「初回=全部fresh」と明示ログ。
2. `**過去draft除外**`: ``.cache/preorders/drafts` + `data/seeds/preorder-pages``の題(base正規化)集合と突合し、`**既ドラフト作は増加分から落とす**`。
3. 出力: ``classified.json`` (or 前段のfresh jsonl)を増加分のみに絞る + 件数ログ「fresh N / 過去draft除外 M / 実処理 K」。

※このscriptが無ければ手動で同等を実施し、**その場でscript化してcommit** (NEVER最終項)。

A. 楽天予約パイプライン (= 主柱)

1. `python scripts/_rakuten-preorder-harvest.py # 6サブジャンル×発売日降順→未来～当月全量(数分)`
- 1b. `python scripts/_preorder-increment.py # ★A0=増加分に絞る(prev差分+過去draft除外)`。飛ばすな
2. `python scripts/_preorder-classify.py # ①続巻/②新作者既知/③新作者新規/④途中巻頁無し/skip`→★生成後、title_kanaに漢字含むドラフト=捏造→hold(NEVER)。全巻回収の先行巻ヨミも楽天由来のみ。
3. `python scripts/_preorder-apply-zokkan.py # ①→種4自動追加(ゲート:slug実在/巻番号/series_key逆引き)`→`promote --only <touch>`→`reflect --only <touch>`→`push` (①は確認不要で即出せる)
4. `python scripts/_preorder-gen-preview.py new1a # ②ドラフト生成(→.preview-data)`
`python scripts/_preorder-gen-preview.py new1b # ③(著者マスタ新規はヨミ=楽天仮)`
`python scripts/_preorder-gen-midfill.py # ④(キャッシュ全巻回収成立分のみ)`
 → preview索引再構築 → push → ★ユーザ確認(段階: ①→②→③→④の順で1クラスずつ)
5. 確認GO後: `python scripts/_preorder-promote-drafts.py --class new1a 等`
 → `data/seeds/preorder-pages/(git恒久保管庫=promote合流結線済・フルpromoteで消えない)`
 → `data/manga.v2(即公開)` → `reflect --only <last-promoted>` → `push`
 ※種2への正式INSERTは月次蒸留時。それまでpreorder-pagesが恒久化を担う。
 - 分類の保留/不備は`docs/production-diagnostics/preorder-triage.tsv`に自動記録。
 - ②③生成時に**キャッチ/詳細/ジャンルも埋める**なら skill enrich-catch-synopsis を続けて実行。

A2. ②③品質ゲート (= 2026-07-06 実戦で確立。previewに出す前に必ず)

****楽天予約の題は信用しない****。入口=★**タイトル分離器`scripts/\_preorder\_title\_lib.py`** (題/巻数/副題の三分解。末尾(N)・第N巻・N巻・中間(N)副題型[三ツ星レシビ]・裸数字・その六かな数詞・上下巻・直結数字はsuspect格下げ[サンダー3保護/鬼平128照合])。分類器/生成器は結線済み=剥がさない。previewに出したら以下を検査(全部実際に踏んだ混入):

1. ****続巻の誤新作化****: 分離器vol ≥ 2 or suspect ≥ 2 $\rightarrow$ 既存頁一意一致+巻連続(max+1..+3)なら種4 / 一致なし=1巻から無い $\rightarrow$ 保留(単巻先行登録禁止)
2. ****上下巻****: ペア=1頁統合(上=v1,下=v2・題から上下除去) / 下巻単独=保留(上巻の全巻回収要)
3. ****scope外****: 特装/限定版・アンソロ・傑作選/再編・コンビニ本(★題でなくimprint判定=集英社リミックス/プラチナコミックス/Gコミックス)・画集/ガイド/BOX・雑誌/別冊・アメコミ翻訳・(仮)
4. ****Zodスキーマ検証****: `loadAllManga`でskip 0確認(year_ended/authors.role/demographic enum=shounen/shoujo…。索引はPython製で検証なし=「検索に載るが404」の既知クラス)
5. ****slug品質****: ★**正規装置`\_slug\_kana\_lib.make\_slug`を使う** (2026-07-09 ユーザ指摘=pykakasi再発明禁止)。janome分かち+***`data/seeds/katakana-english.yml` 貪欲辞書変換** (サマーブレンド $\rightarrow$ summer-blend/デュエルマスターズ $\rightarrow$ duel-masters/ビューティーポップ $\rightarrow$ beauty-pop)+ヘボン(は=wa/長音保持)。**英語綴りが出ない語は辞書に追加して強化** (カタカナ $\rightarrow$ 英単語)。`\_preorder\_draft\_lib.make\_slug`は装置に委譲済。rename時は**made lists+rakuten-kana-pending+staging三点同期**
6. 作者の法人クレジット(Magica Quartet/バンダイナムコ型)は既存DB慣行どおり正当=触らない
7. ★**全巻に発売日+書影が付いているか検査** (2026-07-09 ユーザ指摘): 特に④midfillの**回収先行巻**は要注意=・発売日: 予約巻はharvest、**回収巻は種2(db-v2 volumes.release_date)から引く** (捨てるな)。種2未収載のみ空(NDL/enrichで後補完)。・書影: ★**構築禁止** (2026-07-09 実害=cabinet/バス/サフィックス`\_1\_2`/拡張子`.jpg|.gif`はISBNから推測不可・404)。**実URLのみ**= harvest cover(予約巻) $\rightarrow$ `covers.jsonl.gz` (covers seed) $\rightarrow$ **楽天API `largeImageUrl`** (`real\_cover()`)の順。無ければNone(cover harvest/enrichで後補完)。・gen-preview/gen-midfillは`\_preorder\_draft\_lib.real\_cover()`で結線済み。生成後`release\_date`/`cover\_url`の欠け+**構築URL(`/cabinet/{isbn[-4:-1]}/`)混入**を必ず数える(>0なら原因調査)。

B. NDL新着回収 (= 従来コア)

1. python scripts/_distill_daily.py --discover # 窓=★前回実行月の前月~当月(動的・取りこぼさない)
2. python scripts/_distill_daily.py --plan # 差分レポート+カーソル更新
3. 新規掲載可 $\rightarrow$ ai-todo.jsonl記入 $\rightarrow$ --emit(preview先行) ※詳細は従来手順(下の旧手順参照)
 - ・NDLはNDC付与が納本後のため**未来は取れない** (未来=Aの楽天が担当)。

C. NDL照合キュー消化 (= A裁定「漏れない仕組み」・毎回実行)

```
python scripts/_verify-kana-pending.py --limit 200
```

- ・`rakuten-kana-pending.jsonl`のpendingを古い順にNDL by-ISBN照合。
- ・一致 $\rightarrow$ confirmed / 不一致 $\rightarrow$ `kana-mismatch.tsv` (slug直しの人間判断へ) /

****NDL未収載 $\rightarrow$ pendingのまま残る=漏れない****。

- ・比較は巻番号・上下巻ヨミ差(「~1」「ジョウカン」)を許容(2026-07-06 偽陽性4 $\rightarrow$ 0実証)。
- ・★**不一致の裁定型** (2026-07-14 15件実戦):

****偽陽性****=NDL側の巻マーカー`(001)`/副題有無/スペース・中黒差=対応不要。

****実修正型****=①頁kana末尾に巻数読み(イチ)混入 $\rightarrow$ 剥離 ②slug誤読(魔眼=ma-me型) $\rightarrow$ ヨミ基準でrename

③****NDL改題型****=増補改訂で実題が変わったのに楽天旧題のまま(アスペルガー $\rightarrow$ 自閉スペクトラム症)。NDL by-ISBN=ground truthで題/kana/slug更新。

D. 締め: カレンダー/新刊データ更新

```
python scripts/_build-calendar.py data/manga.v2 data/calendar <当月YYYY-MM> # 本番フル
python scripts/_build-calendar.py .preview-data/manga public/calendar <当月> #
preview(★srcはpreview自身。本番+ALLOWフィルタだとpreview限定ドラフトが落ちる=2026-07-06実害)
```

- ・本番R2へ即時反映したい時: 変更月JSON+manifest+beyond.jsonをPUT(姫松対応の手順)。通常は週次のr2-sync overlayが運ぶ。
- ・previewのカレンダーは**ページ実在フィルタ必須** (subsetなのでフィルタ無し=リンク切れ)。

報告形式

A: 収穫N件(月分布)/①種4追加/②③④ドラフト数+保留・B: 新着N/欠落M・C: 確定/不一致/残pending・D: カレンダー月別巻数。

旧手順の詳細(B系の worksheet 記入規律)

- ・ `is\_manga` / `slug` (ヘボン・勝手命名禁止) / `genres` (closed) / `demographic` / `catch` / `synopsis` (60-120字・ネタバレ無) / `tags` (確信のみ)。
- ・ Layer1: 楽天booksGenreIdがdemographic裏取り(001=少年/002=少女/003=青年/004=レディース/001021=BL)。
- ・ 発売直後はcaption空が普通→ゲートが保留にする=正常(捏造して通さない)。caption供給後の日次で自動的に通る。

関連

- ・ 後退蒸留=`\_distill\_backward.py <年>` / preview管理=skill test-deploy / エンリッチ=skill enrich-catch-synopsis

◆ ★prevの意味(2026-07-10 取りこぼし22件事故 → 同日script化で解消)

- ・ `preorders-prev.jsonl` = **最後に「処理」した時点のfull** (最後にharvestした時点ではない)。処理せず件数確認だけした日をprevにすると、その日の増加分が永遠にスルーされる。
- ・ ★**更新は`\_preorder-increment.py --commit-prev`のみ** (手動copy禁止)。incrementが①full退避を毎回上書き(stale穴封鎖)
- ②絞り済みLATESTへの二重実行をhash検知でabort
- ③処理完了後のcommit-prevでfull→prev昇格+prev.bak退避、まで面倒を見る。runbook手順10。

◆ ★ライブ回収の束ねルール(2026-07-10 明日もいい日=スピンオフ混同事故)

- ・ **束ねは正規化題の完全一致** (副題ブロック込み)のみ。title先頭一致は禁止(本編とスピンオフが同じ書き出しのなろう系で誤マッチ)。
- ・ 束ねた後に**巻番号の連続性を検査** ((1)が2つ/系列飛び=別シリーズ混在signal)。
- ・ ★**著者集合は判定根拠にしない** (ユーザ裁定:
スピンオフで作画が増えるとは限らない=同一人・減る・総入替すべてある。著者軸=分裂の根本原因
[[clustering_unit_is_series]])。
- ・ kana安全網32字の長題誤hold(不貞の子=基底40字なろう系)は**是正済** (2026-07-10): `clean\_kana` が基底題の機械読み長と整合(比0.6~1.5)なら32字超でも通す。副題汚染(読み長より膨らむ)は依然hold。gen-preview/midfill結線済。

diff-deploy

差分反映して (= 差分ビルドエンジン)

name: diff-deploy

description:

差分反映して=変更ページだけ部分ビルド→本番R2へ選択PUT(数分)。コードドリフト時は自動abort→週次蒸留へ誘導

トリガー語: **「差分反映して」**。データ修正(per-case/日次蒸留等)を**本番R2まで数分**で出す軽量ルート。
週次蒸留(フル~3h)との使い分け: **コードが変わったらフル**・データだけなら差分。

実行

```
python scripts/_deploy-differential.py --dry # まず計画(検出結果)を確認
python scripts/_deploy-differential.py # marker→HEAD の data/manga.v2 差分を自動反映
python scripts/_deploy-differential.py --only a,b,c # 明示指定(SRC stem)
```

所要 ≈ 部分ビルド数分 + PUT数秒。

エンジンの安全機構(理解して使う)

1. **コードドリフトガード**: 前回フルビルド以降に
app/manga・app/layout.tsx・components/・lib/・next.config.ts・package.json 等が変わっていたら**abort**(exit 4)→「週次蒸留して」が必要。部分ビルドHTMLが参照するチャンクが本番に無い事故の封鎖(buildId固定が前提)
2. **選択同期**: PUTするのは対象頁(manga/<内部slug>.html/.txt)+本番索引3本(ルートキー)のみ。部分ビルドのホーム/一覧はsubsetデータ汚染物=**絶対同期しない**設計
3. 削除ymlはR2 DELETE / 5MB索引ガード / 3000頁超はabort(フル推奨)
4. **edge cache purge**: worker /api/purge(PURGE_TOKEN認証、.env.local R2_PURGE_TOKEN)で対象URL即時失効
5. marker(.cache/prod-deploy-marker.json)を成功時に更新。**週次蒸留完了時は code_commit/data_commit 両方をビルド時commitに更新すること** (weekly-distill skill 参照)

NEVER

- ・ドリフトabortを--onlyで回避しようとしな(チャンク欠落で本番白画面になる)
- ・トリガー無しで自発実行しない(本番ルールは週次と同じ)
- ・ホーム/一覧/カレンダー等 非mangaファイルを手でPUTしない

事後

- ・疎通出力(疎通 n/3 OK)を確認・ユーザに 更新頁数/削除数/purge結果 を報告
- ・新規頁が index guard(schema検証)で out に無い→abort(exit 7)。先に頁を直す

display-bug-triage

表示不具合の切り分け (= スクショ / 「～がおかしい」報告を受けたら)

name: display-bug-triage

description: 「表示がおかしい」報告の切り分け手順。環境特定→キャッシュ→stale生成物→データ実体の順で1往復診断

今日の実例で各1往復に短縮できた型: カレンダー・K幽霊・生slug表示・灰色書影・臨場の別作化け。

手順(上から順)

◆ 0. どの環境を見ているか(URLで確定)

- ・ `mangal-preview.pages.dev` = テスト(.preview-data push・**反映15-20分・subset**)
- ・ `mangal-r2.shuichi0725.workers.dev` = 本番(週次蒸留/差分反映のみ更新)
- ・ 本番の報告 = 「旧ビルドの内容」の可能性を最初に疑う(直近の修正はテストにしか無い)

◆ 1. キャッシュか実体か

- ・ 本番HTML=edge最長1日、★**JSON/アセット=edge最長7日** (ASSET系 s-maxage=604800。2026-07-13実害=カレンダー/コーナーstockが1週間前のまま) → `?v=xxx` を付けて再現するか確認。stale確定なら worker `/api/purge` で即時失効(R2_PURGE_TOKEN、_deploy-differential.py step6と同機構)
- ・ preview は push 後15-20分待ったか・追いpushでビルドcancelされていないか

◆ 2. ★stale生成物クラスか (= 頻出の真犯人)

生成物は再生成しない限り古いまま。対象と再生成コマンド:

```
| 生成物 | 症状例 | 再生成 |
|-----|
| public/calendar | 旧slug参照で別作品表示(K幽霊)・生slug・月が古い | `_build-calendar.py` |
| public/data/*-stock.json | 書影灰色(slim未展開)・本番に無い作品 | `_gen-corner-stocks.py` |
| public/data/anniversaries等 | 周年/豪華版が古い | `_gen-corner-auto.py` |
| 本番索引(data/) | 消した頁の亡霊が一覧/新刊に出る(レンガ型) | `_build-list-index.py` data/manga.v2 data` |
| .cache/isbn-page-index.json | 存在チェックの誤答 | `_exists.py --build` |
```

◆ 3. 環境仕様の「正常」を誤報しない

- ・ **preview のカレンダー/コーナーで大半が照合失敗するのは正常** (索引がsubset)
- ・ 索引の疎通確認URLは**ルート直下** `/manga-list-index.json` (/data/ ではない)
- ・ 旧カバー巻の書影無し(1980年代等)は楽天に画像が無い=正常

◆ 4. データ実体(ここで初めてyamlを見る)

- ・ 該当頁 dump(editions×volumes×ISBN帯×日付) → percase-fix skill の型分類へ
- ・ ISBNの正体確認は `python scripts/\_lookup.py --isbn ...` (external-data-access skill)

◆ 5. UIコード

- ・ flex内の画像伸縮=コンテナに `self-start` +ボタンはカード欄外、の前例(LikeButton)
- ・ client componentはHTMLに文言が出ない(周年等)→grep不発は異常ではない。JSON配信200を確認

報告様式

「環境=どこ / 真因=上のどの層 / 直したもの / ユーザが確認する場所」の4点で返す。

enrich-catch-synopsis

キャッチ/詳細エンリッチ (= トリガー「エンリッチして」「キャッチ詳細作って」)

name: enrich-catch-synopsis

description: エンリッチして=2巻以上はキャッチ+詳細+ジャンル、1巻はジャンルのみ、材料なしはGemini同定(2026-07-14 定義)。全巻紹介文から役割分担で生成。何度も再発明してきた工程の正本

****キャッチ=検索・一覧に出る惹句 / 詳細(synopsis)=作品ページのあらすじ**。**

両者が似た文になる事故が再発してきた(2026-07-06 ユーザ指摘)ため、役割定義と生成規律をここに固定する。
あわせて同じ材料からジャンル・要素タグも付与する(この一連は過去に何度も実施→この skill が正本)。

★何をどこまで付けるか (= 2026-07-14 ユーザ裁定・エンリッチの定義)

```
|対象| 付与するもの |
|---|
|**2巻以上**+材料(楽天caption)あり| キャッチ+詳細+ジャンル(フル) |
|**1巻のみ**+材料あり| **ジャンルのみ** (キャッチ/詳細は書かない) |
|材料なし(楽天にcaption無し/商品無し)| Gemini同定でジャンルのみ(任意・下記)/それも不可なら**空のまま** (捏造しない)+欠落表 |
```

- ・「楽天でジャンルとれる」の実体 = 楽天が直接くれるのではなく、**captionを材料にAIがmaster32から選んで `genres\_provisional: true` を立てる** (楽天から直接来るのは分野系サブジャンルだけ)。
- ・★**Gemini同定** (2026-07-13 実証・genre:other撲滅で407作適用): 楽天に材料が無い古い作品は `python scripts/\_gemini-genre-probe.py` (gemini-3.1-flash-lite・15RPM・429即中断・再開可)で 題+著者+年から同定。採用ゲート=**known=true × confidence high/medium × master32検証**の3段。art_book/non_manga判定はFLAG(掲載境界)へ、unknown/lowは据置。無料枠~500req/日(PT深夜リセット)。

Step1: 材料収集(全巻の紹介文を読む)

```
python scripts/_enrich-captions.py --missing --src .preview-data/manga --live # 欠け頁を自動抽出
python scripts/_enrich-captions.py --slugs a,b,c --live # 指定頁
```

- ・楽天itemCaptionを**予約キャッシュ→liveの順で全巻分**集める(1.2s/req・429即中断)。
- ・★1巻だけでなく**在る巻ぜんぶ読む** (2026-07-06 ユーザ裁定)。ただし書く内容の縛りは下記。
- ・出力 `cache/enrich/materials.jsonl` (AI生成バッチの入力)。

Step2: AI生成の規律 (= キャッチと詳細を書き分ける・最重要)

材料(全巻captions)を読んだ上で:

```
|| キャッチ | 詳細(synopsis) |
|---|
|役割| 一覧で足を止めさせる**フック** | 作品の**設定と物語**の説明 |
|長さ| **20~40字** | **60~120字** |
|視点| 感情・魅力・引き (「~する物語」等の説明調を避ける) | 主人公・世界設定・物語の起点(1~2巻の範囲) |
|語り| 体言止め/呼びかけ可・煽りすぎ禁止 | 平叙文・客観 |
|禁止| 詳細の短縮コピー (**同じ文の長短2種は不可**) | 最新巻の展開・結末(ネタバレ) |
```

- ・★**丸写し禁止** (verbatim撲滅): captionは材料であり、要約・言い換えて書く。
- ・★後半巻のcaptionは「作品のトーン・ジャンル理解」にだけ使い、**文面には1~2巻の範囲しか書かない**。
- ・楽天にも材料が無い作品(実測: 欠けの~27%)は**空のまま** (捏造しない)+欠落表報告。

Step3: ジャンル・要素の付与(同じ材料から)

- ・**master 32キーから選ぶだけ** (data/genres.yml が正本。新語創作・英語混入禁止 [[ai_genre_closed_vocabulary]])。
- ・1~4個。該当なしは空(otherに落とさない)。**genres_provisional: true を必ず立てる** (AI由来マーク)。
- ・demographic は楽天サブジャンル画像(少年/少女/青年/レディース)が既にあれば触らない。

Step4: 適用

- ・**previewドラフト頁(予約②③④等)**: ymlへ直接書く(ドラフト自体がソース)。
 - ・**本番既存頁**: 適用スクリプトで manga.v2へ純粹追加(空欄のみ埋める)+ 変更slugを「反映して」。
- 永続化seed=`catch-ja.json` (既存)+ `synopsis-slug-ja.json` (2026-07-06新設・promote結線済=空欄fill)。上書き禁止・純粹追加のみ。
- ・100件/batchで処理し、`applied=N, missing=0, overwrites=0` を毎バッチ確認(月次蒸留の保護策と同じ)。

過去の実績・教訓(re-inventしない)

- 2026新刊蒸留: 3段fill+AI enrich(verbatim撲滅・最終巻丸写し禁止はここ由来)。
- 楽天あらすじ→ジャンル: 教師あり+較正+2パス救済で6,638作付与([[genre_from_rakuten_story_plan]])。
- 実測(2026-07-06): キャッチ/詳細欠けの~73%は楽天に材料あり(1巻紹介文はサンプル8/10)=「楽天に無いから空」とは限らない、必ずStep1で突合する。

external-data-access

外部データ照会 (= 楽天・NDL・キャッシュ)

```
---
name: external-data-access
description: 楽天/NDL/キャッシュの照会は必ず _lookup.py
から。レート事故(429/IP遮断)とキャッシュ理解の往復を根絶する
---
```

鉄則

- ・**照会はず `python scripts/\_lookup.py` ** (キャッシュ層→--liveの順が内蔵)。楽天/NDLのlive呼び出しコードを**その場でコピペ再実装しない** (endpoint/header/レートの正は _lookup.py に封じ込め済み)
- ・**1.3秒/req 厳守・429=即中断** (NDLは回復するので慌てず1時間単位で休ませる。並列probeで悪化させた前科あり)
- ・**NDL不在≠不存在** (BL・小出版はNDL収録が弱い=不測ノ恋情の教訓)。实在確認の一次手段は**楽天live(outOfStockFlag=1)**
- ・大量照会の前に「キャッシュで済むか」を必ず先に確認(下の資産マップ)

使い方

```
python scripts/_lookup.py --isbn 9784091204417 # キャッシュ層のみ(即答+delta1-3分)
python scripts/_lookup.py --isbn A,B,C --live # 複数=delta1パス共有→無い分だけlive
python scripts/_lookup.py --title "うる星やつら" --max 20 --live
python scripts/_lookup.py --creator "ひおあきら" --title "宇宙戦艦ヤマト" # ★作者束縛NDL(古典/同名多発の全巻回収)
・★★`--creator` (作品名+作者名でNDL SRU束縛)**=ユーザが手作業でやっていたNDL Search→TSVエクスポートの代替(2026-07-08)。`--title` だけでは楽天liveのみでNDLを叩かず、松本零士版/2199等の人気同名作に埋もれる(ひおあきら版ヤマトの教訓)。**版(seriesTitle)ごとに巻を束ねて表示・著者複数取得** (アンソロ/原作+作画の判定にそのまま使える)。古典・アンソロ・教育系の全巻/版回収はこれが本命。
・★★APIに無く標準WebFetchも弾かれるサイト** (潮出版社usio.co.jp等の公式・静的ページ)は `python scripts/_tinyfish.py fetch <URL>` (無料Fetch)。e-hon等セッション必須の検索は無料Fetch不可=有料Browser領域(要ユーザ承認)。詳細=**skill tinyfish**
```

キャッシュ資産マップ (= どのファイルに何が入っているか。★往復根絶の核心)

```
| ファイル | key | 入っているもの | ★入っていないもの |
|-----|-----|-----|
|.cache/isbn-title-map.json (37万) | ISBN13 | **題のみ** | 日付・書影・価格・著者 |
|.cache/rakuten-isbn-delta.jsonl (830MB) | 行内isbn | **full item** : salesDate/largeImageUrl/itemPrice/author/publisherName/seriesName/itemCaption/booksGenreId | 在庫切れで未収穫の巻 |
|data/seeds/covers.jsonl.gz (30万) | **isbn13/cover_url** (キーマン注意) | 書影URL | それ以外 |
|.cache/isbn-page-index.json (25万) | ISBN13 | 描画中の本番頁slug | ★鮮度注意=大変更後は `_exists.py --build` |
|.cache/volgap-ndl.jsonl | slug | NDL全巻正データ(巻抜け調査の土台) | — |
|.cache/db-v2.sqlite | — | 種2(series/editions/volumes)。series_key逆引き | 種2に無い巻(それが種4の領域) |
```

live仕様の要点 (= _lookup.py 実装済み。修正時のみ参照)

- ・楽天: `openapi.rakuten.co.jp/.../BooksBook/Search/20170404` + **Referer/Origin header必須** (無しやapp.rakuten直=400)。applicationId+accessKey+**outOfStockFlag=1**+formatVersion=2
- ・楽天Kobo(電子版・書影補完用): `openapi.rakuten.co.jp/services/api/Kobo/EbookSearch/20170426` 同じReferer/Origin/レート。title+巻でlargeImageUrlが取れる(紙の欠け巻補完=skill kobo-covers)。noimage除外必須
- ・NDL SRU: `ndlsearch.ndl.go.jp/api/sru` recordSchema=dcndl。**一般語のtitle単独クエリはtimeout** (「臨場」型)→creator束縛を優先・timeoutはスキップ続行可。
- ・★★NDLページングは大物で必須** (2026-07-05実害): 1リクエスト最大200件。数百版ある作(009/おそ松/バカボン)は `startRecord=1,201,401...` で総件数(numberOfRecords)まで**全ページ取得しないと原版を静かに取りこぼす**。1頁だけ取る実装は禁止。ループ=`while start<=total: start+=200`。
- ・★★dcndl:BibResourceの断片割れ** (2026-07-05): レコードが2要素に割れ原版のISBNとvol/dateが泣き別れる型あり。「isbnのみ断片」を直前の「title/vol/date断片」に縫合するmerge_fragments処理を入れる(band-intruder-swapが手本)。
- ・書影CDN構築URL: `thumbnail.image.rakuten.co.jp/@0\_mall/book/cabinet/<ISBN下4桁(チェックディジット除く)>/<ISBN13>.jpg?\_ex=200x200` (在庫切れでも取れる。使用前にHEADで200確認)

大量ハーベスト(数百件超)をする時

- ・逐次追記(1件ずつflush)+再開可能(done-set)+429即中断、の3点セットを必ず入れる(_ndl7_fetch.pyが手本)

- ・事前にユーザへ件数と所要を予告

gemiini-genre-audit

ジャンル検品 (= トリガー「ジャンル検品して」「Gemini検品して」 / 裁定は「**検品裁定して**」。2026-07-14 ユーザ設計)

name: gemini-genre-audit

description: ジャンル検品して/Gemini検品=本番のprovisionalジャンル・要素(~25,000頁)をGeminiでブラインド検品。無料枠(429)まで回して蓄積→不一致だけ裁定。試し読みharvestと並走可の常設アイドルジョブ

★「**検品裁定して**」=溜まった不一致(`--report`→genre-verify-disagree.tsv)を上位モデルがまとめてレビューし、下の採用ゲートを通った分だけ適用する合図(運転=Sonnet/裁定=上位モデル、の分業の後半)。

本番に出ている**AI由来(provisional)のジャンル・要素**には題名だけから幻覚した誤りが混ざる(実例: パワフル☆まぜごはん=短編集なのに「グルメ」)。Geminiに書誌だけ渡して(=現ジャンルを見せない**ブラインド**)同定させ、現行と突合して誤りを炙り出す。

対象と順序 (= scriptが自動決定・対象リストはClaudeが作る=このscriptが正本)

- ・`genres\_provisional: true` かつ genres 非空の本番頁(~25,396 @2026-07-14)
- ・順序: ①catch/synopsis空の頁(=題名だけで付与された疑い濃) ②残りslug昇順(端から全件。人気順禁止)

運転 (= 429まで回す・再開可能・並走OK・★Sonnet運転前提)

★**運転(起動・再開・--report)は安いモデル(Sonnet)の新しいセッションで回す** (2026-07-14 ユーザ裁定: 判断はscript側にあり、AIは起動と報告だけ。上位モデルの長大セッションで回すと起動のたび高コスト)。

上位モデルが要るのは裁定だけ (不一致tsvのレビュー)=溜まってからまとめて依頼する。

python scripts/_gemini-genre-verify.py # queue先頭から429まで(background推奨)

python scripts/_gemini-genre-verify.py --report # 突合レポート+不一致tsv

- ・**429=日次quota(~500req/日, PT深夜=JST16時リセット)で自動中断**。翌日そのまま再実行で続きから。
- ・★**試し読みharvest等と並走可** (Gemini API vs BookLive/TinyFishはホストもレート制限も別物)。

「やることない時」の常設ジョブ: 試し読みループとこれを同時にbackgroundで流すのが基本形。

- ・★quotaは`\_gemini-genre-probe.py` (genre:other残の同定)と共有。**probeの残queueが先**、検品は後。

裁定と適用 (= 自動では直さない)

1. `--report` → `docs/production-diagnostics/genre-verify-disagree.tsv` (不一致+タグ不適合)
2. AIが不一致を目視レビュー (Gemini側の幻覚もあるので鵜呑み禁止。怪しければ魚でWeb裏取り=[[method_ai_generate_plus_webverify]])
3. 採用ゲート: known × confidence high/medium × master32検証 ×
現行がprovisionalの時だけ (trusted/手動は触らない)
4. 適用 = 頁genres更新 + `genre-rakuten.yml` に修正エントリ追記(後勝ちでshadow) + 索引`--update` + commit/push
5. bad_tags(要素の不適合)も同レビューで剥がす(themesから除去)

報告形式

`照会済N(+今回n) / 一致A / 部分P / 不一致D / unknown U` (--reportの出力そのまま)

関連

- ・材料なし頁のジャンル新規付与=skill enrich-catch-synopsis(Gemini同定の項) / genre:other撲滅=同probe
- ・master32厳守=[[ai_genre_closed_vocabulary]] / 1件のバグ=型と疑う=[[feedback_one_bug_means_a_class]]

idle-run

アイドル運転 (= トリガー「アイドル運転して」 / 停止「やめて」。2026-07-14 ユーザ設計)

name: idle-run

description: アイドル運転して=手すき時間の常設2本柱(Gemini検品連鎖+試し読みharvest)を無限ループでbackground起動。
「やめて」で成果を無駄にせず即停止、同じ一言で続きから再開。Sonnet運転前提

やることがない時間に回す常設ジョブのセット。 **時間指定はしない** : 起動→(勝手に走る)→「やめて」で即停止→
別作業→また「アイドル運転して」で続きから。全ループが逐次保存なので停止の損失は最大でも走行中1バッチ(~5分)。

起動 (= 2本を同時にbackgroundへ)

```
bash scripts/_idle-tameshiyomi-loop.sh # ①試し読み(無限・バッチごとcommit+push)
python scripts/_gemini-genre-probe.py && python scripts/_gemini-genre-verify.py # ②Gemini連鎖(429で自然停止)
```

- それぞれ **run_in_background** で別タスク**として起動し、**タスクIDを控えて報告** (=「やめて」で使う)。
- ②はquota(~500req/日・JST16時リセット)で~40分で止まるのが正常。①は対象枯れ/連続エラーまで走る。
- 2本は別ホスト(Google API / BookLive+TinyFish)なので**干渉ゼロ・並走が基本形**。
- ★起動時に直近の Deploy Preview run(GitHub Actions API・無認証)のconclusionを1回見て、failureならユーザに一言報告(デプロイ失敗の見逃し防止=2026-07-13の実害から)。

停止 (= 「やめて」)

- 控えたタスクIDを **TaskStop** で kill**(2本とも)。commit/jsonl済みの成果は全部残る。
- 停止後に現在地を1行報告: 試し読み=`--stats` / Gemini=wc -l .cache/gemini-genre/*.jsonl`。`

再開

- また「アイドル運転して」。両ループとも冪等(done集合/dedup)なので続きから。

NEVER / 注意

- ★**Sonnet運転前提** (このskillの起動・停止・報告に判断は不要。上位モデルの長大セッションで回さない)
- 上位モデルが要る作業はここに混ぜない: 検品不一致の裁定(gemini-genre-audit)・試し読み保留の裁定(tameshiyomi-harvest)は**溜まってからまとめて別途依頼**
- ループscriptを同時に2重起動しない(git push競合)。起動前に既存タスクの有無を確認

セット構成 (= 将来増やせる)

現在: ①試し読みharvest ②Gemini検品連鎖(probe→verify)。

候補: Kobo書影resume / NDL照合キュー消化 — 追加時は「逐次保存・自然停止・冪等再開」の3条件を満たすこと。

関連

- 各柱の正本: tameshiyomi-harvest / gemini-genre-audit / enrich-catch-synopsis(Gemini同定)

isbn-dup-cleanup

ISBNダブリ潰し (= 同ISBNが複数ページに載る事故の解体)

name: isbn-dup-cleanup

description:

ISBNダブリの続き=本番ページ間の同ISBN重複を潰す(検出→層別→判定→適用→検証)。R1-R7(2026-07-07)で確立した正本

トリガー語: 「ISBNダブリの続き」「ISBNダブリ潰して」。進行状態=memory [[isbn-dup-cleanup-state]]。

NEVER (最重要)

- **機械判定は必ず全ログを目視してから適用**(毎回誤りが出る。R4は自動裁定を却下し手動表、R5はoverride6件)
- **部分重なりdedupは固有巻が消える**(page-dedup=頁skipなので、drop側≡canonical側の包含を確認してから)
- **著者不一致ペアをdedupしない**(homonym防御。集合完全一致=同一物理本の時だけ例外)
- **canonical選定に注意**:

傑作選/セレクト/外伝/新装レーベル(CPC等)を本体より優先させない(あおそら家族セレクト/ダイヤのA外伝の誤判定例)

- 巻の帰属判定は本籍sid題との**完全一致**で(substringはスピンオフ題≡本編題ですり抜ける=西風の旅団事故)

手順

1. 検出: `python scripts/\_audit-isbn-dup-pages.py` (~1分) → docs/production-diagnostics/isbn-dup-pages.tsv
2. 層別: `\_isbn-dup-triage.py` (集合一致系) / `\_isbn-dup-r5.py` (汎用判定器=基底題canonical+隠れ本+題系譜)
3. **判定基準**(クラスタごと):
 - **隠れ本あり**(メンバーsidの自ISBNがunionに無い=number-dedupが新版/続シリーズを隠している) →
 - **分割**(merge-exceptions block)
 - **lossless家族**(隠れ本ゼロ×題系譜つながる×同著者) → **dedup**(page-dedup+alias301)
 - 両頁同題で本籍も曖昧 → per-case(外部確認: Wiki/NDL/楽天delta著者)
4. 適用: `\_isbn-dup-apply.py` (auto.json→page-dedup+alias+_redirects+changelog) / blockはmerge-exceptions.ymlへ
5. 反映: `\_reflect-targeted.py --only <生存stem> --drop <drop stem> --push`
6. 検証: 再監査で件数減 + 分割頁が自分の本だけ持つか + `.cache` のjournal的ログ確認

構造知識 (なぜ起きるか)

- DUP_PAGE の正体 = find_related のグループmergeが**メンバーsidごとに同じunion頁をN枚出力** + **number-dedup(最古優先)**が新しい版/続シリーズを隠す(タッチ完全版と同型)
- 種4 qid注入(作者QID)/本籍無しISBN → 種4 seed を疑う
- ゲーム同題群(VP/スターオーシャン/FE/X-MEN型)の解き方:
Wikiのコミカライズ一覧×楽天delta著者×レーベル(エニックススーパーコミック劇場=アンソロ)で
正規コミカライズ各頁+アンソロ1頁(series-merge renumber) に再編。VP前例=2026-07-07
- 過去の一括dedup(2026-06の335件)にも誤りがある(VP土方本編が丸ごと消えていた)=canonicalの中身を疑ってよい
- ★**R1の同ISBN集合dedup(19群)**も後で誤りが出た*: BOYS BE 2nd season(別作品全20巻)を「集合一致」でdropしたが、実は本物20巻が種2に埋もれ1st33冊が両頁に二重出力されていた=2ndがサイトから消失(2026-07-07ユーザ発見)。*
- **同著者ランチャイズの続編(2nd season/L/next等)**は集合一致でも別頁*。dropする前にdrop側SRCの_skey sidの実巻がcanonicalと本当に同一ISBNか(種2で直接)確認。似た危険=ちび本GC/スゴ盛ultra等の姉妹シリーズ

関連道具

- 個別調査: `python scripts/\_isbn-dup-case.py <stem1,stem2,...>` (sid/本籍/楽天題の一覧)
- 照会: `python scripts/\_lookup.py --isbn/--title [--live]` (必ずこれ経由=external-data-access)
- renumber統合: series-merge.yml merge_keys+renumber:true(代表巻は(sid,番号)単位=2026-07-07精緻化済)

kobo-covers

Kobo書影補完 (= トリガー「Koboして」「書影補完して」「Kobo続けて」)

name: kobo-covers

description: Koboして/書影補完して=楽天Kobo電子版の書影で紙の欠け巻を補完。装丁目視ゲート必須(電子が別装丁なら弾く)。resumable

書影欠けの紙巻を、楽天Kobo(電子版)APIの書影で埋める。装丁が紙と一致する時だけ採用する目視ゲートが肝。釣りキチ三平/サザエさん/ゴルゴ13等で実証(2026-07-05)。母集団=軽症組(表紙あり×ISBN欠け)約2,450作。

大原則

- ・★★電子版の装丁が紙と違う作品は弾く★★(= Koboは「デジタル大全」「完全版」等の別装丁で再出版されることがある。サイボーグ009=講談社デジタル大全の無地表紙・花のあすか組=新装アートで REJECT した実例)。誤書影は誤ISBNより悪い([[cover_source_affiliate_only]] [[feedback_accuracy_is_the_goal]])。
- ・★★題+巻番号の完全一致のみ★★候補化(スピンオフ/同題別作を吸わない。BiNGO!=葉芝真己版とコロコロ松村版の混在を発見した型)。
- ・楽天API=★★必ず`scripts/\_lookup.py`相当のヘッダ/レート(1.2-1.3s・Referer/Origin)★★。Koboエンドポイント=`openapi.rakuten.co.jp/services/api/Kobo/EbookSearch/20170426`([[external_data_access]]参照)。

手順 (= バッチループ)

機構 = `scripts/\_kobo-covers.py` (git追跡)。covers seed = `data/seeds/covers.jsonl.gz` (isbn13→cover_url純粹追加、promoteが書込直前に充填)。

1. **prepare** (欠け巻数上位N作をKobo照合+比較ペアDL):

```
python scripts/_kobo-covers.py --prepare 30
```

- ・preview(.preview-data=軽症組)から欠けISBN巻降順でN作。Kobo全ページ照合し題+巻一致の書影を収集。
- ・各作`.cache/kobochek/<slug>-kobo-vN.jpg` (Kobo候補) + `<<slug>-paper.jpg` (既存紙表紙) をDL。一致なしはdone記録しskip。

2. **目視判定** (★私がやる=画像Read):

- ・各作の kobo画像 と paper画像 を Read で並べ、**同一装丁シリーズか**判定。紙表紙が同edition内に無ければ `--anypaper` で別版から代表を落として比較。
- ・ACCEPT=装丁一致 / REJECT=別装丁 (done記録して次へ)。

3. **apply** (合格slugをcovers seedへ):

```
python scripts/_kobo-covers.py --apply slug1,slug2,...
```

4. **反映**: touchedを `\_reflect-targeted.py --only ... --push` (skill reflect-targeted)。

5. **検証**: 反映後`残欠け`が減ったか数字確認。REJECT/発見(同題混在等)は docs/production-diagnostics のworklistへ。

ヒット率の実測

軽症組でKoboヒット~35%、さらに装丁目視で絞られ**実補完は推定800-900作**。電子未配信(ドカベン=水島新司拒否/学習まんが/貸本)は埋まらない=無理しない。1回10-20作。

罣

- ・楽天cacheに題があっても書影が`noimage` のことがある→除外必須([[rakuten_out_of_stock_flag]] [[rakuten_cover_data_asset]])。
- ・ISBN無し巻(プレISBN古典)はKoboでも埋まらない=母集団外 (ISBN有欠けのみ対象)。
- ・藤子㊟系は頁側の装丁次第でACCEPT/REJECTが割れる=必ず個別目視(まんが道=REJECT/魔太郎がくる=ACCEPT)。

long-job-ops

長時間ジョブの運転法

name: long-job-ops

description:

長時間ジョブ(build/promote/sync/harvest)の運転法。生存確認・監視の絞り方・ハング判定・Windows/シェルの落とし穴

鉄則

1. **起動直後に生存確認してから turn を終える**(12時間死んでいた同期事故の教訓)。ログが無言でも `Get-Process python | Select CPU` でCPU時間が伸びていれば稼働中(ハッシュ照合等は10-25分無言が正常)
2. 60秒超のジョブは `run\_in\_background` + **Monitor**。事前に1行で予告
3. Monitorの通知は**節目だけに絞る**(1万頁ごと・致命エラー・完了)。2分毎の進捗通知は過多→作り直す
4. Monitorのフィルタは**失敗シグナルも網羅**(成功語だけgrepすると crash が沈黙=進行中と区別つかない)。buildなら `attempt 3 of 3|Export encountered|Build error` を必ず入れる
5. **完了判定は成果物とログ末尾**で行い、プロセス終了を待たない:
promote=ログ最終「art-books」/manga.v2ファイル数→kill([[promote_hangs_on_exit_windows]]). next build=`✓ Exporting (2/2)`+out/manga件数。node/pythonの居座りはWindows仕様
6. 実行中に出力ディレクトリを覗かない(ロック競合)

監視コマンド型(実証済み)

- ・単発完了通知: Bash `run_in_background + `until <条件>; do sleep 20; done``
- ・節目Monitor: ログから現在値を取り「1万単位が変わった時だけecho」+失敗grep+プロセス消滅でbreak
- ・★`sleep N; コマンド`の直列チェーンはツールにブロックされる→untilループかMonitorへ

シェル/Windowsの落とし穴 (= ops-pitfalls 統合)

- ・**複数行pythonは必ず .cache/_xxx.py にファイル化して実行**。bash heredocはJSX/バックスラッシュ/引用符で壊れる(今日も2敗)。パス文字列はf-string+`{ROOT}`でraw文字列の` \ U`事故回避
- ・コンソール出力の日本語化は cp932 表示だけの問題(処理は正常)。`sys.stdout.reconfigure(encoding='utf-8')`を全スクリプト冒頭に
- ・PowerShellは `&&` 不可(5.1)・ヒアドキュメント閉じ `'|` は列0
- ・**wrangler kv は既定でローカル(miniflare)を見る**→実データは `--remote` 必須(30分空転の実績)
- ・wrangler secret/deploy 直後は**伝播ラグ数十秒**→即テストで403/旧応答でも数十秒待つて再試験
- ・R2実体の確認は wrangler より **S3 API(boto3+.env.local認証)**が確実
- ・edge cache確認は `?v=<適当>` でバイパス。HTML s-maxage=86400
- ・git commit をコマンドチェーン末尾に置かない(前段failでも走る)。tsc→OK確認→commit の順で分離

★destructiveコマンドとシェル変数 (= 2026-07-06 全消し事故から)

- ・`python scripts/\_promote-bulk-v2.py --only "\$VAR"` 型は **必ず `[-n "\$VAR"] &&` を前置**。
\$VARが空だとフルモード(=manga.v2全削除→再生成)に落ちた実害(ガードは実装済みだが二重防御)。
- ・**フルpromote等の長時間処理をBashのtimeout付きフォアグラウンドで直接実行しない**
(途中killで「削除済み・書きかけ」の中途半端状態になる)。run_in_background で回す。
- ・復旧手順(実証済): manga.v2=種2+seedの純派生。seedがcommit済みなら `python scripts/\_promote-bulk-v2.py` (フル)で全成果込み完全復元→本番索引+カレンダー再生成+当日修正頁のスポット検証。
- ・索引フル再生成のskip平常値=**~-923**(genre other 920=既知未解決)。0でないと慌てず「増えたか」で見る。

monthly-distill

月次蒸留して

```
---
name: monthly-distill
description: 月次蒸留して=MADB取込→フルpromote→enrich→AI
fill. Phase0前提確認→差分報告→Goサイン必須の大工程(~3時間+)
---
```

トリガー語: **「月次蒸留して」**(完全一致)。CLAUDE.md「月次蒸留 protocol」が正本——**必ず併読**。ここは運用の実体メモ。

大原則

種1/種2/種3は壊さない。純粹追加only。上書き/削除検出=即abort+報告。**Phase1の差分報告→ユーザGoサイン受領までPhase2に進まない**。

実パイプライン ([[monthly_distill_real_pipeline]])

1. Phase0 前提確認 = ★`python scripts/\_monthly-phase0.py` (2026-07-10 script化。前提10項+git cleanを機械確認・欠け=exit 1=「対象Xが無いので蒸留できない」とユーザ報告して終了。目視チェックリストで代替しない)
2. MADB差分取得: サイト月次CSV(cm101+cm504のみ更新。cm103/104/105は2024-11凍結=再DL無駄) + GitHub全件JSONのdateModified比較で訂正回収
3. `clean-madb-seed` → `\_build-series-v2` → `\_populate-v2` (temp) → **`\_distill-incremental-merge`** (series_key突合の安全純粹追加・INSERT only)
4. `python scripts/intake.py --run` (roles→merge→seed4→matcher v9→v13→v14→adult_us→trailing→foreigndrop→promote。matcher~20分・promote~110分。**Windows: promote完了後プロセス居座り=ログ最終行/ファイル数で判定しkill**)
5. enrich: AniList再フェッチ(毎回。凍結gapは毎月増える)・synopsis和訳delta(分散WF→_apply-synopsis.py 純粹追加→commit)・作品QID(QLever)
6. AI fill batch(種3 diff→100件/batch・JST報告・commit毎)
7. **月次サニティ監査**: _coverage-audit(前月差分flag)・_audit-volume-numbering(AUTO_FIXED急増=新型signal)・_furgana-audit・_audit-title-eq-author・_audit-foreign-editions・publisher新キーflag・**特装検出(新刊特装→special-edition seed追記=ベルセルク43型)**・_audit-solo-truncated
8. **stale生成物の再生成**: _build-calendar / _gen-corner-stocks / _gen-corner-auto / 本番索引
9. 最終summary(全件数+削除0確認+次月予測)

罣

- ・再登録の別MADB-ID二重化(虚構推理vol23型)→ISBN dedup+監査
- ・種4は触らない(手動add only)。retire hygiene だけ(MADB追いつき分の除去)
- ・本番R2への反映は別途「週次蒸留して」

★成功判定 (= 完了主張の前にこの数字を全部言えること)

- ・Phase0 exit 0 / **Goサイン受領の発話引用**(無しにPhase2へ進んだら違反)
- ・種1/2/3 とも **削除0・上書き0** (各取込ログの `applied=N, missing=0, overwrites=0`)
- ・種2 series数 = **増加のみ** (減少=即abort案件) / promote後 manga.v2 ≈ **66k+ files** (激減=事故)
- ・tsc/vitest **green維持** (赤転落=abort) / サニティ監査の flag件数(0でなくてよい=報告する)
- ・どれかが言えない=完了していない

new-manga-register

新規追加 / 新刊入れて

name: new-manga-register

description: 新規追加/新刊入れて=新規マンガをテスト先行で登録(順番固定protocol・捏造/単巻先行の再発防止)

トリガー語: 「新規追加」「新刊入れて」「この漫画を載せて」等。

流れは**テスト先行が例外的に許される唯一のルート**: preview生成→ユーザ確認→GOで本番化。

順番固定 (= 2026-07-02 ユーザ裁定。CLAUDE.md 「新規登録 protocol」)

1. **全巻回収が先** — 巻Nを発見したら title+creator で

NDL全巻+楽天全巻(outOfStockFlag=1)を回収。 **単巻先行登録は禁止**

2. **題の確定** —

NDL題×楽天題を突合。不一致=調査、不明=ユーザ報告。 **勝手命名は絶対禁止**。 slugは確定題+確定ヨミから一度だけ生成

3. **ヨミの確定** —

題ヨミ=NDLタイトルヨミ。 著者名+ヨミ=NDL典拠/楽天。 不明=報告して待つ。 役割(原作/作画)不明もデフォルトで埋めない

4. **一括登録** — 全巻+必須メタ(title/kana/romaji/authors/year/status/demographic/genre≥1)が揃ってから

5. **enrichは登録後** — 1巻基点でネタバレ無しのcatch/synopsis。 genreはclosed

vocabulary(master32キー、trusted無ければprovisionalマーク)。 最終巻あらずじ丸写し禁止

6. **作れないものは作らない** — 埋められない項目=空+欠落表で報告。 必須すら確定できない作品=登録保留リスト(載せない)

実装ルート

- ・少数=daily/backward distill の worksheet フローに乗せる(_distill_daily / _distill_backward)
- ・生成後: 検証(Zodミラー/slug衝突/日付pad) → preview投入(test-deploy skill) → ユーザ確認 → GO後に「反映して」で本番系列へ

索引ガードとの関係

authors/genres/year/kana

非空要求は「埋める圧力」になる→**埋めるな、保留にせよ**(チェック通過のための捏造が最悪)。

percase-fix

per-caseは正 (= イアラ式。ユーザが作品名+Wiki/NDLリンクを出したら)

name: percase-fix

description: 作品名+リンク(Wiki/NDL)を渡されたら per-case

で版/巻/ISBNは正(イアラ式)。汚染除去・版分離・巻補完・variants の選び方まで

NEVER (最重要)

- **推測やダミーで埋めない**([[feedback-accuracy-is-the-goal]])。不明=止めて報告
- **単一ソース+解釈で上書きしない**。Wiki×楽天×NDL の2ソース以上一致で確定
- NDL/楽天 live は **1.2-1.3秒/リクエスト**厳守。429=即中断(逐次保存しつつ)
- NDL不在=不存在と断じない(BL/小出版はNDL収録が弱い=不測ノ恋情の教訓)。楽天live(outOfStockFlag=1)が一次手段
- canonical に「現ページの現状」を凍結しない(汚染ごと凍結する=うる星の失策)。**外部権威(Wiki)から作る**
- 価格を表示に出さない(内部ソートのみ可)
- ★**種4に qid を書かない** (種2のqid=作者QID→作者の全作品に巻が注入される。うしおとら→藤田14作事故 2026-07-07。series_keysで十分)
- ★**種4の release_date はフル日付でも必ず引用符**(`'1986-09-18'`)。裸だとYAMLがdate型に解釈されpromoteがクラッシュ)

2026-07-07 promote側で封鎖済の罫(知っておく)

- 混入巻除去(volume-exclude)+真巻登録(種4)のセット: 旧来は「混入巻が番号占有/偽日付タイ/ISBN無し幽霊」の三重の壁で真巻が弾かれた→全て封鎖済(こち亀vol1)。同時に書いてよい
- edition-override頁の出版年: 確定巻と既存年が非交差なら自動再計算(上杉謙信)。交差時は連載年保持(タッチ)
- renumber統合の代表巻=(sid,巻番号)単位(1sidに別書籍複数でも取りこぼさない=VPアンソロ第3集)

調査手順

1. `python scripts/\_ledger.py <slug>` で操作履歴+holes / `python scripts/\_exists.py --title/--isbn` で本番存在
2. 現ページ dump: editions×volumes×ISBN帯(prefix)×日付 → 混在/欠け/幽霊を分類
3. Wikipedia 書誌節を WebFetch(巻別ISBN・発売日が権威)。ISBN-10→13変換は check digit 再計算
4. 楽天キャッシュ照合: `.cache/isbn-title-map.json` (題)→ 日付/書影は
`.cache/rakuten-isbn-delta.json`。無ければ楽天live(openapi.rakuten.co.jp+Referer/Origin
header、_complete-edu-gaps.py の search() 参照)
5. NDL SRU(title+creator)。「臨場」等の一般語title単独クエリはtimeoutする→creator束縛+失敗スキップ続行

型別の直し方(どの seed を使うか)

```
| 型 | 症状 | 直す場所 |
|---|---|---|
| 単純巻抜け | 欠番・ISBN確定可 | **種4** volumes-supplement.yml(series_keys=既存ISBN→db-v2逆引き) |
| 幽霊巻 | ISBN無し・実在しない巻番号 | **edition-overrides.json** (实在巻だけで再構築) |
| 版混在(奇子型) | 別版ISBNが枠に滲む・日付逆行 | 小規模=edition-overrides /
大物・刷タブ要=**edition-canonical/*.yml** (versions対応済) |
| 別作混入(Frankenstein) | 別作品の巻が枠に居る | 移設=種4(本来頁keys)+混入頁は volume-exclude or page-dedup |
| スピンオフ不可視 | 同名スピンオフのページが無い |
**merge-exceptions.yml** (id対称block)+series-keep.yml (spinoff旧作drop救済)+種4gap fill |
| 断片重複 | 同ISBNが2頁に | **page-dedup.yml** (drop→canonical) |
| 特装が主枠 | 特装ISBNが通常枠 | **special-edition-fix-redo2.yml** に correction
追加(同一出版社コードのみ。跨ぎはoverride直書きvariants) |
| 書影無し | covers seed に無い | covers.jsonl.gz へ追記(**キーは isbn13/cover_url**。url ではない) |
```

適用と検証

- changelog(volume-gaps-changelog.jsonl 等)に1行 → **反映して** (reflect-targeted skill)
- 検証: 再生成後の頁 dump で 欠0・prefix整合・variants/書影 を数字で確認
- 新規発見の系統バグは検出器化を検討(_audit-solo-truncated.py / _audit-volume-output.py が前例)

productionize-drafts

本番化して (= 確認済み予約ドラフトの本番化。2026-07-09 ユーザ設計)

name: productionize-drafts

description: 「本番化して」 =

確認済み予約ドラフトをpreorder-pages(恒久)へ昇格+preview解放。次の週次蒸留で本番R2公開。トリガー「本番化して」。

ユーザが preview で確認を終えた予約ドラフト(②③④)を、**週次蒸留で本番に載る状態**にする。

狙い: 日次蒸留で溜めたドラフトを恒久保管庫へ移し、**テスト環境を解放して別作業に移れる**ようにする。

いつ使うか

- ・トリガー語「**本番化して**」(完全一致でなくてよい: 「本番化」「preorder載せて」「確認したので本番化」も同義)。
- ・前提: ユーザが preview を**確認済み** (この skill 起動=確認完了の意思表示)。特定作のみなら slug 指定。

やること (1コマンド)

```
python scripts/_preorder-productionize.py # preview上の全予約ドラフトを本番化
python scripts/_preorder-productionize.py --slugs a,b # 指定分のみ本番化(残りはpreviewに据置)
python scripts/_preorder-productionize.py --keep-preview # 本番化するがpreviewにも残す
```

内部で順に:

1. `\_preorder-promote-drafts.py` = 各ドラフトを **`data/seeds/preorder-pages/` (git恒久・フルpromoteで合流)** + `data/manga.v2` (即時データ) へ。`\_preorder\_draft` 注記を除去。**本番既存slug衝突はskip**。
2. 本番索引 `\_build-list-index.py` data/manga.v2 data --update <promoted>` 増分更新。
3. **previewから除去** (テスト環境解放) + preview索引再構築(`--keep-preview`で残せる)。

締め (必須)

```
git add data/seeds/preorder-pages data .preview-data #
★data/manga.v2はgitignore対象=addに入れると失敗してchainが切れる(2026-07-11)
git commit -m "予約ドラフト本番化 N件(preorder-pages恒久・週次で公開)"
git push
python scripts/_exists.py --build # ★ISBN索引再構築(2026-07-14:
これを忘れると次のclassifyが既掲載ISBNをskipできず二重ドラフト化)
・変更slugは`.cache/preorders/last-promoted.json`に出る。commitメッセージに件数を入れる。
・★slug渡しは内部でファイル経由(--slugs-file/--update-file
2026-07-14)=1,000件級もWindowsコマンドライン長制限(WinError206)に当たらない。
・★特定作を本番化から外したい時(ソーサリアン型hold)=対象yamlを一時`.cache/`に退避→実行→戻す。
```

この skill が「やらない」こと (= 誤解防止)

- ・**R2ライブ公開はしない**。preorder-pages/manga.v2(データ+git)まで。**実際に本番サイト(mangal-r2)に出るのは「週次蒸留して」** (フルpromote+R2 sync)。ユーザ設計どおり「溜める→週次でまとめて公開」。
- ・即時に本番サイトへ出したい時だけ、続けて「**差分反映して**」 (diff-deploy=部分ビルド→R2 PUT)。

なぜ preview から除去するか

- ・確認済み分は preorder-pages に**恒久保管**されるので preview に残す必要がない。
 - ・★除去で**テスト環境が空く**=次の日次蒸留(増加分だけ)や別作業がクリーンにできる(ユーザ要望「確認が終わったら別作業」)。
 - ・★増加分ゲート(`\_preorder-increment.py`)が **preorder-pages
- の題を除外**するので、次の日次蒸留で**再カウントされない** (未promoteドラフトの再登場問題を根絶)。

報告形式

- ・本番化 N件 / 衝突skip M件 / preview解放(残 K件) / 「次の週次蒸留で本番公開」を明示。
- ・保留(hold)されていたドラフトは対象外(previewにも出ていない)=そのまま残る。

関連

- ・生成=skill daily-distill(手順5がこの本番化) / 本番R2公開=skill weekly-distill / 即時=skill diff-deploy / per-case反映=skill reflect-targeted(予約ドラフトには使えない=SRC前提)

prodwait-preview

本番待ちをテスト環境へ (= 2026-07-10 確立・script化)

name: prodwait-preview

description: 本番待ちテストに出して=前回週次以降の全変更(本番待ち)を.preview-dataへ一括投入し、次の週次で公開される内容を事前レビュー可能にする。1コマンド(_prodwait-to-preview.py)

トリガー語: **「本番待ちテストに出して」「本番待ち全部テストへ」**(「次の週次の分を見たい」も同義)。
週次蒸留の**事前レビュー**が目的: 次回「週次蒸留して」で公開される変更を、先にpreviewで全部確認できる状態にする。

やること (1コマンド)

```
python scripts/_prodwait-to-preview.py --dry-run # まず件数と日別内訳(何も書かない)
python scripts/_prodwait-to-preview.py --push # 投入+索引+カレンダー+commit+push
```

内蔵(scriptが面倒を見る・手で再実装しない):

1. cutoff=markerのcommit日時 → mtimeで本番待ち頁を列挙
2. **mtime汚染ガード**: >20,000頁=フルpromote(全消し復旧等)の汚染と判断してabort → 実変更の始点を調べ`--since YYYY-MM-DD` (2026-07-06全消し復旧で実際に踏んだ)
3. **preview専用ドラフト温存** (manga.v2に無い確認待ち②③④は消えない)
4. masters6本の差分同期(漏れ=新出版社キー頁404の罫)
5. 索引再構築+**skip診断は内部slugで** (ファイル名≠slugのslug-override頁を誤検出しない)+原因分類。`genre:other`=既知クラス(本番索引も同様にskip)=報告のみで止めない
6. previewカレンダー再生成(src=preview自身)

オプション運用

- **日付絞り** (2026-07-14 ユーザ運用): 投入後「最新巻の発売日が YYYY-MM 以降だけに絞って」= 各頁の全版・versions
- variants込みの最新release_dateで判定し、それ未満のyamlをpreviewから削除→索引+previewカレンダー再生成。**実体(manga.v2/seeds)は無傷**=表示の間引きだけ・週次には全部乗る。preview専用ドラフト(sorcerian等)は特例で残す。

NEVER / 罫

- **push後15-20分・追いpush禁止** (test-deployと同じ)
- 本番R2へは出さない(公開は「週次蒸留して」のみ)
- 週次実行後はmarkerが進むので、次回からcutoffは自動で正しくなる(--sinceは汚染時だけ)
- previewが千頁級になる=正常(1,218頁で実証済)。「セットを絞りたい」時は skill test-deploy(per-case投入)を使う

関連

- per-case/少数セットの投入=skill test-deploy / 公開=skill weekly-distill / ドラフト昇格=skill productionize-drafts

reflect-targeted

反映して (= targeted反映)

name: reflect-targeted

description:

反映して=seed/per-case修正を本番manga.v2+テスト環境へ数分で反映(targeted)。フルpromote(3時間)は使わない

トリガー語: **「反映して」**. per-case修正(数~数百頁)を数分で反映する既定ルート。

★フルpromote(66k再生成~110分+索引)は週次/月次蒸留のときだけ。

NEVER

- per-case にフルpromoteを使わない
- 変更slugをユーザに聞かない(**自分で列挙**: 触った seed の key → 対応slug)
- push後の追いpushをしない(previewビルドがcancelされ「変わらない」の正体になる)

手順

```
python scripts/_reflect-targeted.py --only <SRC-stem,...> [--drop <消すstem,...>] [--push -m "メッセージ"]
```

- `--only` = **manga.v2ファイル名(SRC slug)**。slug-override頁もSRC名で指定。
- `--drop` = 消す頁(page-dedup/non-manga-drop 登録後に)。manga.v2+preview 削除+索引remove まで自動。
- 内蔵: 検証ゲート(slug/kana/date/isbn 不正で push 前停止) • promote --only • 索引
- --update/--remove(本番+preview) • preview同期 • 台帳集約 • push。
- 書影は promote に統合済(別工程不要)。

反映先の意味

- 本番系列 = data/manga.v2 + data/索引(gitのみ。 **本番R2は週次蒸留まで出ない**)
- テスト = .preview-data(pushで自動デプロイ、 **反映15-20分・ 追いpush禁止**)
- preview の頁subsetに無い作品を確認したい時: `cp data/manga.v2/<slug>.yml .preview-data/manga/` + `python scripts/\_build-list-index.py .preview-data/manga .preview-data` + push

罣

- edition-canonical 結線slug(golgo-13/tsuribaka-nisshi/urusei-yatsura)は edition-overrides
- を直しても**canonicalが後勝ち**(reflectが警告する) → canonical側を直す
- 新publisherキーを masters に足したら `preview-data/publishers.yml` にも cp(preview 404の原因)
- 日付は YYYY-MM-DD/YYYY-MM(ゼロ埋め)。全角数字はNFKC

tameshiyomi-harvest

試し読み拾って (= BookLive title_id 収集。2026-07-12 設計、同日★全巻展開を追加)

name: tameshiyomi-harvest

description:

試し読み拾って=BookLiveの試し読みリンク(title_id)を魚で収集しseed化。判断はscript、Allは保留裁定のみ。Sonnet運転前提

MANGALの「試し読み」ボタン用に、BookLiveの作品ID(title_id)を集めて

`data/seeds/tameshiyomi-booklive.json` (= シリーズ単位アンカー) と

`data/seeds/tameshiyomi-booklive-volumes.json` (= ★巻単位の実データ、ボタンが読むのはこっち) に貯める。

収集だけがこのskillの仕事 (ボタンUI/中継/via_cd注入は別作業=このskillでは触らない)。

★全巻展開の発見 (=

2026-07-12、ユーザ「ドラゴンボールなら42巻分取らないとダメよ？」で判明)

BookLiveのtitle_idは**シリーズ/版単位**。product頁の`vol\_no`パスやbviewerのcid末尾3桁を

変えるだけで**同一title_idのまま全巻に到達できる** (TinyFish検索は不要・HEADチェックのみで済む)。

実証済: `title\_id/582763` (チェンソーマン) で `vol\_no/001` → 1巻、`/002` → 2巻。

`bviewer cid=582763\_001/\_002`、`185409\_001/\_025`、`185407\_100` 全部HEAD200。

帰結: **アンカー(シリーズ→title_id)さえ集めれば、巻ごとの追加検索は不要**。

`cid=f"{title\_id}\_{vol:03d}"` を `max(total\_volumes, max\_edition\_volumes)` 分HEAD検証するだけ

(= `\_tameshiyomi-harvest.py --expand`)。TinyFish検索が要るのは**新規シリーズのtitle_id発見時だけ**。

収集するのは title_id(+巻の実在HEAD確認)だけ (= 2026-07-13 ユーザ裁定)

購入ページURLは収集しない。試し読みビューアも購入ページも `title\_id` + `巻番号` から決定的に生成できる:

- ・試し読み: `booklive.jp/bviewer/s/?cid=<title\_id>\_<巻3桁>`

- ・購入: `booklive.jp/product/index/title\_id/<title\_id>/vol\_no/<巻3桁>`

アフィリエイトパラメータもURL生成時(ボタンUI実装側)に付ける。収集の仕事を勝手に増やさない。

NEVER (最重要・全部実害防止)

- ・**title_idを推測や連番で作らない**。scriptの検索+検証を通ったものだけがseedに入る

(= ただし全巻展開の`vol\_no`/cid末尾3桁を変える手法は実証済みの例外、これは検索でなくHEAD検証)

- ・**保留の採用は証拠付きのみ**: `--accept` する前に、候補title_idのBookLive頁の題・著者が

MANGAL頁と一致することをsnippet/fetchで自分の目で確認。曖昧なら保留のまま残す(残すのは正常)

- ・**seedを手で編集しない** (追記はscript経由のみ。壊すと全ボタンが死ぬ)

- ・TinyFishは**無料枠のみ** (Fetch/Search)。Agent/Browserは絶対に使わない(クレジット消費)

- ・1回の**検索(--limit)**実行は100まで。それ以上は分割(TinyFish 150req/分制限)。

★**--expand(HEAD検証のみ)にはこの制限は不要** (TinyFishを一切呼ばない・BookLive直HEAD。★2026-07-13からHEAD8

並列=高速。楽天/NDL/TinyFishの逐次レート則是別ホストなので適用外)

- ・検索失敗/エラーで止まったら**そのまま再実行** (再開可能設計)。リトライ連打しない

手順 (= この順で回す)

```
1. python scripts/_tameshiyomi-harvest.py --stats # 現在地(アンカー数/保留/全巻展開数)
2. python scripts/_tameshiyomi-harvest.py --limit 100 # 新規シリーズのtitle_id発見(検索。人気順)
3. python scripts/_tameshiyomi-harvest.py --expand --expand-limit N # ★アンカー済み全シリーズを全巻展開(検索不要・高速)
4. python scripts/_tameshiyomi-harvest.py --review # 保留一覧
5. (保留の裁定) 各行について:
- 魚で `site:booklive.jp <題>` を検索し直し、候補title_idの頁題がMANGAL題と同一作品か確認
- 確信が持てたものだけ: python scripts/_tameshiyomi-harvest.py --accept <slug>=<title_id>
- 確信なし/BookLiveに無い作品 → 保留のまま放置(それが正しい)
- accept後は忘れず3を再実行(新規acceptしたslugも全巻展開する)
6. commit+push:
git add data/seeds/tameshiyomi-booklive.jsonl data/seeds/tameshiyomi-booklive-volumes.jsonl
docs/production-diagnostics/tameshiyomi-holds.tsv
git commit -m "試し読みharvest: +N件(保留M)/全巻展開+K巻" && git push
```

scriptが保証していること(理解用・いじらない)

- ・対象選定=本番索引の人気順(popularity)上位から未収集・未保留のみ
 - ・採用ゲート=①題の正規化**完全一致**(部分一致は保留) ②著者姓がsnippetに存在
 - ③ビューアURL(`bviewer/s/?cid=<id>\_001`)の**HEAD 200実在検証**。3つ揃わないと自動採用しない
 - ・出力=jsonl純粹追加(証拠snippet・検証日付き)・保留=tsv。再実行で重複しない
 - ・`--expand`=アンカー済み(title_id確定)シリーズについて、`data/manga-list-index.json`の`total\_volumes` / `max\_edition\_volumes`を上限にcid各巻をHEAD検証、200のみ`tameshiyomi-booklive-volumes.jsonl`に`{slug, volume, title\_id, cid}`で純粹追加。
- 既に検証済みの巻はskip(再開可能)。1シリーズ完了ごとに件数を表示

保留の典型と裁定のコツ

- ・**完全一致なし**: 表記ゆれ(ONE PIECE⇄ワンピース等)や巻数付き題。BookLive頁を魚でfetchして題+著者が一致すれば --accept。シリーズ違い(0巻/外伝)を本編と間違えないこと
- ・**複数候補**: 無印と新装版/カラー版が並ぶ。**無印(通常版)のtitle_id**を選ぶ
- ・**候補0**: BookLiveに無い(古書・成年など)。放置

報告形式

`アンカー合計X件(+今回N) / 保留Y件(うち今回裁定でZ件採用) / 全巻展開=W巻(V件シリーズ完了)`

関連

- ・魚の使い方=skill tinyfish / ボタンUI・中継(/go)実装=別途(Claude本体の作業)
- ・将来: シーモアはビューアURL規則が不透明なため、ASP提携後に別途設計

test-deploy

テスト環境に出して (= preview投入)

name: test-deploy

description:

テスト環境に出して=対象頁を.preview-dataへ投入/入替してmanga-previewで確認可能にする(push自動デプロイ・15-20分)

トリガー語: 「テスト環境に出して」「テストに入れて」「preview で見たい」等。

★**「本番待ち全部」を出す時は skill prodwait-preview**(`\_prodwait-to-preview.py --push`

1コマンド=週次の事前レビュー)。このskillはper-case/少数セット用。

手順

1. 入替なら先に空にする: `rm -f .preview-data/manga/\*.yaml` (ユーザが「消した上で」と言った時のみ)
 2. 対象を copy: `cp data/manga.v2/<stem>.yaml .preview-data/manga/` (複数可)
 3. preview索引再構築: `python scripts/\_build-list-index.py .preview-data/manga .preview-data` (~20秒/500頁)
 4. masters が変わっていたら同期: `cp data/publishers.yaml .preview-data/publishers.yaml` 等
- ★**schema(lib/schema.ts)のenum等を変えた時は必ず全masters(demographics/genres/publishers/magazines)をdiff確認**。
- 2026-07-14実害: demographics.yamlからother削除時にpreviewミラー未同期→enum backstopがbuildを止め、preview deploy3連続failure(ユーザにエラーメール)
5. `git add .preview-data && git commit && git push`

NEVER / 罣

- **push後15-20分待つ・追いpush禁止** (前ビルドがcancelされ「変わらない」)
- preview索引はsubset=正常。 **public/ の索引(ルート直下3本)を本番索引に差し替えない** (previewが66k化して壊れる)
- UI変更は .preview-data 不要(コード push だけで preview に出る)
- デプロイ確認は Actions REST API か時間経過。 反映されない時はビルドcancelを疑う
- 本番R2へはこのskillでは**絶対出さない** (週次蒸留のみ)

★previewセット管理 (= 2026-07-06 「元々入っていた漫画が消えない」 事故から)

previewは**「いま確認したいセットだけ」を入れる** (混在=確認ノイズ+件数混乱)。

- セット入替: `.preview-data/manga/` を対象yamlだけにする (旧セットは削除可=本番/ステージングに実体がある)
- ドラフト退避場所: `.cache/preorders/drafts/` (未確認の②③④はここ。previewに出すのは確認する分だけ)
- 入替後は必ず: `python scripts/\_build-list-index.py .preview-data/manga .preview-data` (索引)

+ previewカレンダー再生成(`\_build-calendar.py .preview-data/manga public/calendar <当月>`

★srcはpreview自身=ドラフトも載る)

+ push

- ★カレンダーは二重化済み: public/=preview実在フィルタ版 / data/calendar=本番フル(r2-sync overlayが自動で本番へ)

tinyfish

TinyFish (= WebFetch補完。2026-07-08 導入・2026-07-10 skill化)

name: tinyfish

description: 魚で調べて=TinyFishだけで調査/魚などで調べて=全ソース(_lookup.py→WebFetch→TinyFish)で調査。APIが無い+標準WebFetchが弾かれるサイトの取得(fetch)とWeb検索(search)。無料枠のみ

AIエージェント向けWebインフラ。MANGALでは**「APIが無い+標準WebFetchが返らない/弾かれる」サイトの取得**と、**性質判定のWeb検索** (これはアンソロカ/雑誌か等) に使う。位置づけ=**必須でない便利ツール** (ユーザ評)。

トリガー語 (= 「魚」=TinyFishの愛称。2026-07-10 ユーザ設定)

```
| ユーザの言い方 | やること |
|---|---|
| **「魚で調べて」** |
| **TinyFishだけで調査(search/fetch)。**_lookup.py等の他ソースは使わない(ユーザがTinyFish単独の結果を見たい時) |
| **「魚などで調べて」** | **全ソース調査**=下の「使う順番」どおり(_lookup.py→WebFetch→TinyFish
| のエスカレーション込みの深い調査) |
| 「TinyFishで」 「タイニーフィッシュ」 | 文脈次第(単独指定ならTinyFishのみ) |
| (トリガー無し) | 状況発動=per-case/蒸留中に「APIに無い+WebFetch弾かれ」に当たったら自分の判断で使う |
```

使う順番 (= エスカレーション。いきなりTinyFishに行かない)

1. **NDL/楽天/キャッシュ** = `python scripts/\_lookup.py` (skill external-data-access が正本)
2. **標準 WebFetch/WebSearch** (Wikipedia・大手サイトはこれで足りる)
3. **TinyFish** = 上2つで取れない時だけ(出版社公式=潮出版社usio.co.jp型・小さい版元・JS重の電書ストア=cmoa/BookWalker・Amazon書影/在庫)

使い方 (= scripts/_tinyfish.py。urllib実装・requests不要)

```
python scripts/_tinyfish.py fetch <URL> [<URL>...] # ページ→Markdown本文(最大10URL/回・POST)
python scripts/_tinyfish.py search "<クエリ>" # Web検索→構造化JSON(★GET。POSTは失敗する)
```

コード内: `from \_tinyfish import fetch, search`

- ・ `fetch(urls, live=True)` = キャッシュ無視(ttl=0)。既定はTinyFish側キャッシュ応答。
- ・ `search(query, purpose=...)` = 応答 `{query, results:[{position,title,url,snippet,site\_name,date?}], total\_results}`。purposeで検索意図を補足できる(任意・2000字まで)。
- ・ fetch応答には `results[]` と別に `errors[]` (url/error/status)が出る。エラーは黙って捨てない=件数を報告。

鉄則

- ・ **キー = `.env` の `TINYFISH\_API\_KEY`** (git追跡外。★絶対コミットしない。無ければ即エラーで止まる)
- ・ **無料枠のみ使う**： Fetch/Search=無料(150URL/分)。 **Agent(フォーム操作)/Browser(ステルス)はクレジット消費=使う前に必ずユーザ承認**： e-hon等セッション必須の検索は無料Fetchでは不可=有料Browser領域。
- ・ **一次ソースの序列は変えない**： 書誌の正はNDL/楽天。TinyFishで取った公式ページ情報は「NDL/楽天に無い版情報・発売情報の補完」であり、既存データを上書きする根拠には単独でない(2ソース確認の原則はそのまま)。
- ・ 大量fetch(数十URL超)は他のharvest同様、事前に件数予告+逐次追記+再開可能にする(skill long-job-ops)。

実証済みの型 (= どういう時に想起するか)

```
| 型 | 例 | 使うもの |
|---|---|---|
| 出版社公式にしか無い版/刊行情報 | 潮出版社 usio.co.jp (WebFetch不通を fetch で取得成功) | fetch |
| 「これはアンソロ/雑誌か」の性質判定 | スゴ盛=実話系4コマ雑誌と確定 (Wikipedia+版元ドットコムがsnippetで出る) | search |
| per-case/蒸留で楽天・NDL両方に無い書誌 | 小版元の公式・電書ストア商品ページ | search→fetch |
```

報告形式

- ・ fetch: 取得N/失敗M(errorsの内訳)。search: ヒット要旨+採用したURL。
- ・ 取れなかった時は「無料Fetch不可(セッション必須)=有料Browser領域」かを区別して報告(ユーザ承認マター)。

関連

- ・照会の正本=skill external-data-access(まず _lookup.py) / 事実の記録=memory [[tinyfish_web_fetch]]

volgap-audit

巻抜け仮想 / 巻系監査

name: volgap-audit

description: 巻抜け仮想=残巻抜けを算出(~2分・promote不要)。単巻切り詰め検出(solo-truncated)もここ

巻抜け仮想 (トリガー語: 「巻抜け仮想」)

```
python scripts/_volgap-virtual.py --list
```

本番DBの巻抜けフィルタを仮想再現(未promoteのseed適用後の残欠を~2分で算出)。促されたら件数+内訳(ISBN有無/homonym)を報告。

★方針: ISBN無し・homonym絡みは無理しない(skip)。安全fixのみ。per-caseは percase-fix skill へ。

単巻切り詰め検出 (シャイナ・ダルク型)

```
python scripts/_audit-solo-truncated.py
```

1冊のみ+巻番号≥2 を全DB走査 → docs/production-diagnostics/solo-truncated.tsv に3分類:

- DUP_ELSEWHERE = 断片重複 → page-dedup 最有力
- CACHE_SIBLINGS = 楽天に別巻実在 → 切り詰め濃厚(per-case復元)
- NO_EVIDENCE = **自動fill禁止**。電子先行の紙化(不測ノ恋情型=紙は途中巻のみが正)が正当に混ざる。楽天live(outOfStockFlag=1)で個別確認

巻出力監査(本番yaml側)

```
python scripts/_audit-volume-output.py
```

promote出力の #1欠落/日付矛盾を検出(種2を見る旧監査の見逃し領域)。

前提

索引が古いと誤検出する: 大きい変更の後は `python scripts/\_exists.py --build` (ISBN頁索引) を先に。

weekly-distill

週次蒸留 (= フルビルド・フルアップ)

```
---
name: weekly-distill
description: 週次蒸留=本番フルビルド+R2フルアップ。トリガー「週次蒸留して」。テスト環境で確認済みの変更を本番へ出す
唯一の定期ルート
---
```

トリガー語: **「週次蒸留して」**(完全一致でなくてよい)。
★本番へのアップはこのトリガー(または明示指示)以外で**絶対に自発実行しない**。

NEVER

- ・トリガー無しで本番 build/sync しない
- ・価格の静的表示を含むUIを出さない(grep で `\\|price` 表示確認)
- ・`--prune` を安易に付けない(削除は明示判断)
- ・完了主張の前に疎通確認を飛ばさない

手順

◆ 1. 事前再生成 (= stale生成物クラスを全部焼き直す)

```
python scripts/_build-calendar.py data/manga.v2 data/calendar <当月YYYY-MM> # ★本番フル(r2-syncがこれをoverlay)
python scripts/_build-calendar.py .preview-data/manga public/calendar <当月> # preview版(src=preview自身)
# ★引数なし実行は禁止(2026-07-13実害): 既定out=public/calendarにフル版が書かれ、本番overlay元のdata/calendarは古いまま
# =本番カレンダーが前週のまま stale。しかも生成器は古い月ファイルを消さないの、srcを替える時は out を rm -rf してから。
python scripts/_gen-corner-stocks.py # 三代目/featured stock JSON
python scripts/_gen-corner-auto.py # 周年/豪華版 JSON(66k走査 ~5分)
python scripts/_build-list-index.py data/manga.v2 data # 本番索引(~10分)
  • ai-reviews 等 seed 由来はそのまま(生成不要)。
  • 生成物を commit+push。
```

◆ 2. ★preflight (= 2026-07-10 script化。手動チェックリスト全廃)

```
python scripts/_weekly-preflight.py --fix # FAILが1つでもあればビルド開始禁止(exit 1)
  • 内蔵: コード未コミット検査(2026-07-04実害)/timeout=300/D:空き20GB+/out・.next
  junction再作成(★実体dirは絶対に自動削除しない=中身有りは手動退避を指示)/staging
  junction+masters6+索引3本の同期/生成物鮮度WARN。
  • 下の「ディスク事前確認」の手動PowerShellはpreflightが代替(復旧手順のみ手動参照)。
```

◆ 3. フルビルド (★実測2.5~3.5h、バックグラウンド+Monitor)

★**preflight全通過(exit 0)を確認してから開始**(C:満杯だと終盤のexportコピーがENOSPC死する)。
`$env:MANGAL_DATA_DIR="D: \ mangal-cache \ proddata"; npx next build 2>&1 | Out-File .cache \ weekly-build.log`
・★`staticPageGenerationTimeout=300s` (next.config.ts)前提。既定60sだと重負(home-design=66k全読込)がワーカー競合で3回超過しビルドkill(2026-07-05 home-design-05で発覚・是正済)。
・Monitor は 1万頁節目+「after 3 attempts / Export encountered / Build error」+完了のみ通知(2分毎は通知過多)。
・attempt 1-2 の retry は **コールドワーカーの初回66k読込(warmup)**=正常。300s猶予で温まればリトライ成功。
・終盤(6万頁以降)は重負が残り生成速度が落ちる=正常。完了判定: log 末尾 `✓ Exporting (2/2)` + `out/manga` ファイル数 $\approx$ 頁数 $\times$ 2($\approx$ 132k)。
・★完了後の**居座りnodeをStop-Process**(Windows恒例。file lock解除)。

◆ 3.5 sitemap生成 (build後・sync前)

```
python scripts/_gen-sitemap.py
(out/ に sitemap.xml+分割を書く=syncが拾う。SEO②)
```

◆ 4. R2 同期 (差分PUT)

```
python scripts/_r2-sync.py --bucket mangal-site
  • 起動直後に**生存確認**(python の CPU 時間が伸びているか)。ログ0バイトでもハッシュ照合中は無言(10-25分)が正常。
  • スクリプトが .env.local から R2_* 自動読込・**本番索引 overlay(out/ ルートへ)+5MBガード**内蔵。
  • レイアウト級の変更(全頁共通部)があった週は全量PUT=正常。
```

・★**同期後にJSONのedge cache purge必須**(2026-07-13実害): workerのASSET系は
`s-maxage=604800`=**エッジ7日**。
purgeしないと calendar/* .json ・ data/*-stock.json 等が最長1週間前のまま配信される(ユーザ画面が更新されない)。
purge = worker `/api/purge` (R2_PURGE_TOKEN認証、_deploy-differential.py step6 と同機構)。最低限
`/calendar/manifest.json` + `/calendar/release/\* .json` (当月~3ヶ月+beyond) + `/data/\* .json` を対象に。

◆ 5+6. ★finalize (= 2026-07-10 script化。疎通→marker→manifestをゲート連鎖で1本化)

python scripts/_weekly-finalize.py

- ・内蔵順序: ①ビルド完了判定(log『✓ Exporting』 +out/manga≥120k枚) → ②sitemap存在 →
- ③疎通確認(`\_prod-smoke.py` = 主要頁200/索引ルート直下+5MBガード/\非含有/contact POST) → ④marker更新 →
- ⑤`\_init-pages-manifest.py`。
- ・★**①~③のどれかがFAILならmarkerを書かずabort** (=diff-deployは前回基準のまま安全側)。手動one-liner写経は廃止。
- ・疎通だけ単独で回す時: `python scripts/\_prod-smoke.py` (検証練習は
`--no-post`)。URL正解はscriptに焼き込み済(索引は**ルート直下**=/data/ではない。2026-07-04誤検証の教訓)。

◆ 7. 報告

工程表(ビルド頁数/PUT数/疎通結果)で完了報告。異常は隠さずそのまま。

★成功判定 (= 完了主張の前にこの数字を全部言えること)

- ・preflight exit 0 / out/manga ≈ **132k files** (≥120kがfinalize下限) / R2 PUT数(全量=レイアウト週・少数=データ週)
- ・smoke **PASS 10/10** (FAIL 0) / 索引 **25MB級** (5MB未満=preview索引化事故)
- ・marker `note=週次蒸留` + pages-manifest 初期化ログ
- ・どれかが言えない=完了していない(finalizeがexit 0を返すまで完了報告禁止)

★ディスク事前確認 (= 2026-07-10より `\_weekly-preflight.py --fix` が自動実施。以下は仕組みの説明+ENOSPC復旧手順)

★**out/と.nextは常にD:へジャンクションしてからビルド** (C:は満杯気味・D:に455GB空き)。`out/+ .next`で計10-15GB要る。

手動でやる場合(PowerShell):

```
Get-PSDrive C,D | Select Name,@{n='Free_GB';e={([math]::Round($_.Free/1GB,1))}} # 確認
cmd /c rmdir "C:\Users\shuic\code\MANGAL\out" "C:\Users\shuic\code\MANGAL\ .next" 2>$null
New-Item -ItemType Directory -Force "D:\mangal-cache\weekly-out", "D:\mangal-cache\next-build" | Out-Null
cmd /c mklink /J "C:\Users\shuic\code\MANGAL\out" "D:\mangal-cache\weekly-out"
cmd /c mklink /J "C:\Users\shuic\code\MANGAL\ .next" "D:\mangal-cache\next-build"
```

- ・junction中は `next build` がout/を消す時リンクを触るので、**毎ビルド前にrmdir→mklinkし直す**のが安全。
- ・★ENOSPC復旧(万一C:直ビルドで途中失敗した時): outをD:へ`robocopy SRC DST /E /MOVE` 退避→`cmd /c rmdir`でjunction作成→`.next/server/app`の`.html`/`.rsc`をoutへ手コピー(.rsc→.txt改名・既存skip)で**2h再ビルド回避**できる(2026-07-05実証)。junction除去は`cmd /c rmdir` (Remove-Itemは中身を追ってD:保護に弾かれる)。

備考

- ・★カレンダーは索引と同型の二重化(2026-07-06): public/calendar=preview実在フィルタ版なので、_r2-sync.pyが**data/calendar(本番フル)で自動overlay**する。Step1の再生成はフル/preview両方を回すこと(daily-distill Dと同じコマンド)。
- ・Defender除外は実施済(2026-07-04)。ビルドが異常に遅い時は`Get-MpPreference | Select ExclusionPath`で除外が生きてるか確認。
- ・本番ドメイン mangal-db.com = 紐付け済(2026-07-10 疎通200確認。smokeの既定BASE)。
- ・edge cache(HTML s-maxage=86400)により旧頁が最長1日残る。確認は`?v=`クエリでバイパス。

★ビルド環境の罠(2026-07-12 実害3連発→恒久対処済み。消すな)

・★D: \ node_modules junction 必須**: ` .next`をD:へjunctionすると、ビルド成果物からの`require('react')`がnode_modulesに届かず`Cannot find module 'react/jsx-runtime`で死ぬ。対処=`cmd /c mklink /J D: \ node\_modules C: \ Users \ shuic \ code \ MANGAL \ node\_modules`
(作成済み。消えていたら再作成)。next-build内に旧ビルド残骸があると同エラー→`rm -rf D:/mangal-cache/next-build/\*`
・**buildは必ずStart-Processでデタッチ起動**: ツールのrun_in_backgroundは~10分で親ごとkillされworker巻き添え死。
`Start-Process powershell -ArgumentList "-NoProfile", "-File", "D: \ mangal-cache \ \_wkbuild.ps1" -WindowStyle Hidden`

- **r2-syncも同様にscript

file経由でデタッチ**(`D: \ mangal-cache \ \_r2sync.ps1` 作成済み)。PSの`|Out-File`パイプ直渡しは空ログ即死する

wiki-distill

Wiki蒸留して

name: wiki-distill

description: Wiki蒸留して=Wikipedia書誌(巻別ISBN+発売日)で壊れた長期連載をcanonical復元。①全自動型/②ドカベン式/③現状維持の型判定つき

トリガー語: **「Wiki蒸留して」**。壊れシグナル持ち(帯混入/うる星型/巻抜けworklist)×巻数多い順に、Wikipedia書誌を権威としてページを復元する。釣りキチ三平65巻/スケバン刑事22巻で実証(2026-07-04)。

手順

1. ****対象選定****: docs/production-diagnostics/ の各worklist(band-intruders-manual / urusei-type-manual / volgap374-triageのMERGE・NONE)を合算し、巻数多い順に10~15作/回。canonical既存slugはskip
2. ****実行****:

```
python scripts/_wiki-distill.py --slugs a,b,c --fetch --write
```

 - ・記事raw取得(1s/req・.cache/wiki/にキャッシュ)→書誌ブロックparse→型判定
 - ・①巻別ISBN型 → canonical自動生成(下のゲート全通過時のみ)
 - ・②構成のみ型/③書誌なし → worklist報告(②は後日ドカベン式=Wiki版構成+楽天/NDL充填)
3. ****反映****: ①型のslugを`\_reflect-targeted.py --only ... --push`
4. ****検証****: 各頁で 帯1種化/日付逆行0/巻数=宣言N を数字で確認(達成できない場合は原因を報告)

ゲート(fail-closed・緩めない)

- ・見出し『題』が**頁題とbase一致するブロックのみ**採用(スピンオフ/別編を吸わない=サイボーグ009のBGOOPARTS事故防止)
- ・ISBN-10→13は**checksum再計算** (Wikiのハイフン揺れ耐性)
- ・**宣言「全N巻」と抽出数の一致**必須(取りこぼし=②型へ降格)
- ・**楽天題baseゲート**：cacheに居るISBNの題が1件でも別作ならabort
- ・Wiki自体の誤記対策=楽天gate+反映後の帯/逆行検証の二重網
- ・★**手動救済は判断作業=Fable級のみ** (2026-07-10): 弱いモデル(Opus等)で運転する時は手動救済をせず、「③型なのに記事にISBN有り」をリスト化してユーザ報告で止める(fail-closedのまま)。誤った部分一致canonicalは本番汚染=誤skipより重い。
- ・★**題記号/表記揺れでauto gateが誤skipする作は手動救済** (2026-07-05実証):
楽天題gateはbase完全一致なので、①ハート記号(U+2764 vs ♥U+2665) ②ラテン/カナ(WEED vs ウィード) で別物判定→③書誌なし扱いになる。記事にISBNがあるのに③になったら疑う。救済=該当ブロックのISBNを抽出し、楽天照合を**不変部の部分一致** (銀牙/キャッツ 等、同題別作を弾ける固有語)に替えて手動でcanonical書出。銀牙伝説Weed(全60巻・不変部「銀牙」)/キャッツ・アイ(全18巻・「キャッツ」でシティハンター混入除外)で実証。
- ・★**巻数上位worklistは下るほど自動歩留まりが落ちる**：完結×全N巻×巻別ISBNの好条件作は先に枯れ、残りは連載中(既刊)/記事なし/ISBN無しアンソロが増える。1バッチ①0でも異常でない。ISBN有りなのに③の作だけ手動精査すればよい。

型の実測比率(パイロット7作)

①巻別ISBN 約4割 / ②構成のみ 約4.5割 / ③なし 約1.5割。候補プール=壊れ518作(巻数10+は116作)。

罣

- ・記事の書誌が「新装版のみ」の場合、主版=最多巻ブロック選択が新装を主にしてしまう→反映後検証で頁の帯が全交換されていたら疑う(原版温存が原則)
- ・複数部構成(第1部/第2部)の記事は全N巻表記が部単位のことがある→巻数不一致で自然に②へ落ちる(安全側)
- ・楽天cache無しの古典はgate素通りになりがち→逆行/帯検証を最終網とする